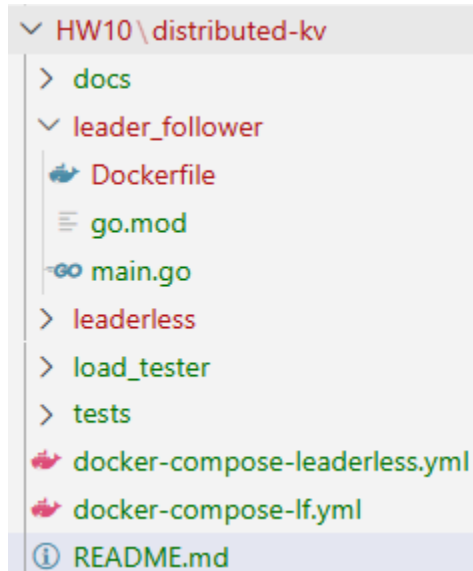


Homework 10 Report

Wilson Neira

Distributed Databases using Replication

Key-Value (KV) Service



How the Leader-Follower system works

There are 5 nodes total: 1 leader and 4 followers.

All writes go to the leader. The leader updates itself first, then replicates to followers.

W=5, R=1: The leader waits for acknowledgements from all 4 followers before returning 201.

A read only needs the leader copy.

W=1, R=5: The leader updates itself and returns as soon as the leader write completes.

Replication to followers continues in the background.

A read contacts all 5 nodes and returns the newest version.

W=3, R=3: The leader returns after the leader plus 2 followers acknowledge.

A read collects 3 copies and returns the newest version.

Logical version numbers are stored with each key-value pair so the newest version can be selected. The local_read endpoint is used only for testing. It reads the local state on a node without the leader proxy path. This helps expose the inconsistency window during propagation.

How the Leaderless system works

There are 5 peer nodes. Any node can receive a write request. The node that receives the write becomes the write coordinator. It writes locally, sends the update to the other 4 nodes, waits for all of them, and then returns 201. A normal read returns the node's own local value. This means a client can hit a stale node during the write propagation window. That behavior is intentional for the assignment because it shows the inconsistency window.

Tricky parts

- In leader-follower, $W=1$ returns early while replication continues, which creates a stale-read window.
- In leader-follower, followers proxy normal reads to the leader so that public reads can go to any instance.
- In leaderless, any node can act as write coordinator, so stale reads are easier to expose during propagation.
- Version numbers are necessary to compare records and return the newest value for quorum reads.

Error handling

- Empty keys return 400.
- Missing keys return 404.
- If the required number of acknowledgements is not reached, the write returns 500.
- Internal peer failures are caught and handled so the system can still expose quorum behavior.

AI Usage Statement

AI assisted with:

- brainstorming repo structure
- drafting starter code
- drafting test and plotting script structure
- helping outline documentation

I reviewed and understood all the code before using it. I understand:

- how writes propagate in the leader-follower system
- how quorum reads and writes are implemented
- how the leaderless coordinator works
- how the tests expose the inconsistency window
- how the load tester measures stale reads and latencies

Leader-Follower Database

python -m pip install pytest requests

Testing Consistency

docker compose -f docker-compose-lf.yml up --build -d

PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW10\distributed-kv> curl http://localhost:8001/health

```
StatusCode      : 200
StatusDescription : OK
Content         : {"node_id":"leader","role":"leader","status":"ok"}

RawContent      : HTTP/1.1 200 OK
                  Content-Length: 51
                  Content-Type: application/json
                  Date: Sun, 05 Apr 2026 01:19:20 GMT

                  {"node_id":"leader","role":"leader","status":"ok"}

Forms           : {}
Headers         : {[Content-Length, 51], [Content-Type, application/json], [Date, Sun, 05 Apr 2026 01:19:20 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 51
```

PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW10\distributed-kv> curl http://localhost:8002/health

```
StatusCode      : 200
StatusDescription : OK
Content         : {"node_id":"f1","role":"follower","status":"ok"}

RawContent      : HTTP/1.1 200 OK
                  Content-Length: 49
                  Content-Type: application/json
                  Date: Sun, 05 Apr 2026 01:19:22 GMT

                  {"node_id":"f1","role":"follower","status":"ok"}

Forms           : {}
Headers         : {[Content-Length, 49], [Content-Type, application/json], [Date, Sun, 05 Apr 2026 01:19:22 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 49
```

```
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW10\distributed-kv> curl http://localhost:8003/health
```

```
StatusCode      : 200
StatusDescription : OK
Content         : {"node_id":"f2","role":"follower","status":"ok"}

RawContent      : HTTP/1.1 200 OK
                  Content-Length: 49
                  Content-Type: application/json
                  Date: Sun, 05 Apr 2026 01:19:22 GMT

                  {"node_id":"f2","role":"follower","status":"ok"}

Forms           : {}
Headers         : {[Content-Length, 49], [Content-Type, application/json], [Date, Sun, 05 Apr 2026 01:19:22 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 49
```

```
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW10\distributed-kv> curl http://localhost:8004/health
```

```
StatusCode      : 200
StatusDescription : OK
Content         : {"node_id":"f3","role":"follower","status":"ok"}

RawContent      : HTTP/1.1 200 OK
                  Content-Length: 49
                  Content-Type: application/json
                  Date: Sun, 05 Apr 2026 01:19:22 GMT

                  {"node_id":"f3","role":"follower","status":"ok"}

Forms           : {}
Headers         : {[Content-Length, 49], [Content-Type, application/json], [Date, Sun, 05 Apr 2026 01:19:22 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 49
```

```
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW10\distributed-kv> curl http://localhost:8005/health
```

```
StatusCode      : 200
StatusDescription : OK
Content         : {"node_id":"f4","role":"follower","status":"ok"}

RawContent      : HTTP/1.1 200 OK
                  Content-Length: 49
                  Content-Type: application/json
                  Date: Sun, 05 Apr 2026 01:19:28 GMT

                  {"node_id":"f4","role":"follower","status":"ok"}

Forms           : {}
Headers         : {[Content-Length, 49], [Content-Type, application/json], [Date, Sun, 05 Apr 2026 01:19:28 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 49
```

Unit Test

Write a unit that tests the Leader database for consistency:

- The unit test send set to the Leader Node.
- After the Leader acknowledges the write, read from the Leader. That should return consistent data.
- After the Leader acknowledges the write, read from a Follower. That should return consistent data.

pytest tests/test_leader_follower.py -s

```
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW10\distributed-kv> docker compose -f docker-compose-lf.yml up --build -d
#26 [lf-f4] resolving provenance for metadata file
#26 DONE 0.0s
[+] up 11/11
✓ Image distributed-kv-lf-f3      Built
✓ Image distributed-kv-lf-f4      Built
✓ Image distributed-kv-lf-leader  Built
✓ Image distributed-kv-lf-f1      Built
✓ Image distributed-kv-lf-f2      Built
✓ Network distributed-kv_default  Created
✓ Container lf-f2                 Created
✓ Container lf-f4                 Created
✓ Container lf-f3                 Created
✓ Container lf-f1                 Created
✓ Container lf-leader             Created
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW10\distributed-kv> pytest tests/test_leader_follower.py -s
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW10\distributed-kv
plugins: anyio-4.8.0, locust-2.43.3
collected 2 items

tests\test_leader_follower.py ..

===== 2 passed in 3.18s =====
```

docker compose -f docker-compose-lf.yml down

```
===== 2 passed in 3.18s =====
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW10\distributed-kv> docker compose -f docker-compose-lf.yml down
[+] down 6/6
✓ Container lf-f1                 Removed
✓ Container lf-leader             Removed
✓ Container lf-f3                 Removed
✓ Container lf-f2                 Removed
✓ Container lf-f4                 Removed
✓ Network distributed-kv_default  Removed
```

Leaderless Database

Testing Consistency

docker compose -f docker-compose-leaderless.yml up --build -d

```
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW10\distributed-kv> curl http://localhost:8101/health
```

```
StatusCodes      : 200
StatusDescription : OK
Content           : {"node_id":"n1","status":"ok"}

RawContent       : HTTP/1.1 200 OK
                  Content-Length: 31
                  Content-Type: application/json
                  Date: Sun, 05 Apr 2026 01:28:50 GMT

                  {"node_id":"n1","status":"ok"}

Forms            : {}
Headers          : {[Content-Length, 31], [Content-Type, application/json], [Date, Sun, 05 Apr 2026 01:28:50 GMT]}
Images           : {}
InputFields      : {}
Links            : {}
ParsedHtml       : mshtml.HTMLDocumentClass
RawContentLength : 31
```

```
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW10\distributed-kv> curl http://localhost:8102/health
```

```
StatusCodes      : 200
StatusDescription : OK
Content           : {"node_id":"n2","status":"ok"}

RawContent       : HTTP/1.1 200 OK
                  Content-Length: 31
                  Content-Type: application/json
                  Date: Sun, 05 Apr 2026 01:28:51 GMT

                  {"node_id":"n2","status":"ok"}

Forms            : {}
Headers          : {[Content-Length, 31], [Content-Type, application/json], [Date, Sun, 05 Apr 2026 01:28:51 GMT]}
Images           : {}
InputFields      : {}
Links            : {}
ParsedHtml       : mshtml.HTMLDocumentClass
RawContentLength : 31
```

```
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW10\distributed-kv> curl http://localhost:8103/health
```

```
StatusCodes      : 200
StatusDescription : OK
Content           : {"node_id":"n3","status":"ok"}

RawContent       : HTTP/1.1 200 OK
                  Content-Length: 31
                  Content-Type: application/json
                  Date: Sun, 05 Apr 2026 01:28:51 GMT

                  {"node_id":"n3","status":"ok"}

Forms            : {}
Headers          : {[Content-Length, 31], [Content-Type, application/json], [Date, Sun, 05 Apr 2026 01:28:51 GMT]}
Images           : {}
InputFields      : {}
Links            : {}
ParsedHtml       : mshtml.HTMLDocumentClass
RawContentLength : 31
```

```
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW10\distributed-kv> curl http://localhost:8104/health
```

```
StatusCode      : 200
StatusDescription : OK
Content         : {"node_id":"n4","status":"ok"}

RawContent      : HTTP/1.1 200 OK
                  Content-Length: 31
                  Content-Type: application/json
                  Date: Sun, 05 Apr 2026 01:28:51 GMT

                  {"node_id":"n4","status":"ok"}

Forms           : {}
Headers         : {[Content-Length, 31], [Content-Type, application/json], [Date, Sun, 05 Apr 2026 01:28:51 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 31
```

```
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW10\distributed-kv> curl http://localhost:8105/health
```

```
StatusCode      : 200
StatusDescription : OK
Content         : {"node_id":"n5","status":"ok"}

RawContent      : HTTP/1.1 200 OK
                  Content-Length: 31
                  Content-Type: application/json
                  Date: Sun, 05 Apr 2026 01:28:54 GMT

                  {"node_id":"n5","status":"ok"}

Forms           : {}
Headers         : {[Content-Length, 31], [Content-Type, application/json], [Date, Sun, 05 Apr 2026 01:28:54 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 31
```

Unit Test

For the Leaderless database, write a unit test that exposes the inconsistency window:

- The unit test writes a key-value pair to a random node. That node is now the Write Coordinator.
- Within the update time window test read (i.e. send get messages) to other nodes. These returns should be inconsistent.
- After the Coordinator acknowledges the write, read from the Coordinator. That should be consistent.
- After the Coordinator acknowledges the write, read from another node. That should be consistent.

```
pytest tests/test_leaderless.py -s
```

```

PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW10\distributed-kv> docker compose -f docker-compose-leaderless.yml up --build -d
#26 [11-n3] resolving provenance for metadata file
#26 DONE 0.0s
[+] up 11/11
✓ Image distributed-kv-11-n3      Built      25.0s
✓ Image distributed-kv-11-n4      Built      25.0s
✓ Image distributed-kv-11-n5      Built      25.0s
✓ Image distributed-kv-11-n1      Built      25.0s
✓ Image distributed-kv-11-n2      Built      25.0s
✓ Network distributed-kv_default  Created    0.1s
✓ Container 11-n5                 Created    0.7s
✓ Container 11-n2                 Created    0.7s
✓ Container 11-n4                 Created    0.7s
✓ Container 11-n3                 Created    0.7s
✓ Container 11-n1                 Created    0.6s
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW10\distributed-kv> pytest tests/test_leaderless.py -s
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW10\distributed-kv
plugins: anyio-4.8.0, locust-2.43.3
collected 2 items

tests\test_leaderless.py ..

===== 2 passed in 3.00s =====

```

docker compose -f docker-compose-leaderless.yml down

```

===== 2 passed in 3.00s =====
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW10\distributed-kv> docker compose -f docker-compose-leaderless.yml down
[+] down 6/6
✓ Container 11-n3                 Removed
✓ Container 11-n1                 Removed
✓ Container 11-n2                 Removed
✓ Container 11-n5                 Removed
✓ Container 11-n4                 Removed
✓ Network distributed-kv_default  Removed

```

Load Testing

Run all required load-test ratios

The following ratios on:

3 Leader-Follower configs

1 Leaderless config

Ratios:

1% writes / 99% reads

10% writes / 90% reads

50% writes / 50% reads

90% writes / 10% reads

`python -m pip install -r load_tester/requirements.txt`

How the Load Test Generator Works

My load test generator uses a small hot set of keys instead of a very large random keyspace. That means the same keys get read and written again and again within short time windows instead of spreading all requests across too many different keys. This matters because the assignment wanted reads and writes to be local-in-time, meaning that a read should often happen soon after a write on the same key. My generator encourages that naturally by reusing a limited set of keys and continuously choosing from them during the test. Because of that, the system has more chances to show stale reads or version differences if there is an inconsistency window. So, the generator is not just random traffic, it is intentionally designed to make consistency problems easier to detect by clustering activity around the same keys close together in time.

Run LF strategy W=5,R=1

Start LF cluster:

```
docker compose -f docker-compose-lf.yml up --build -d
```

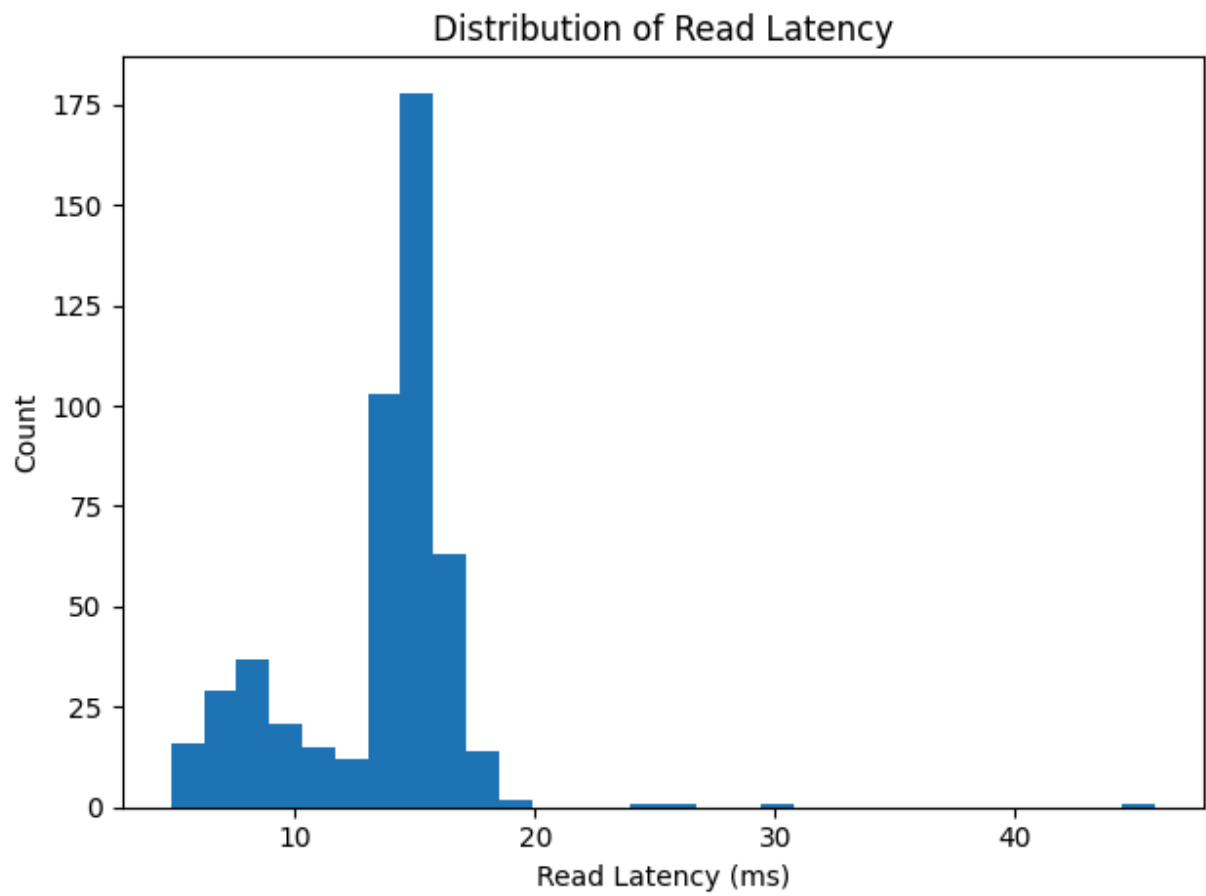
Start LF cluster:

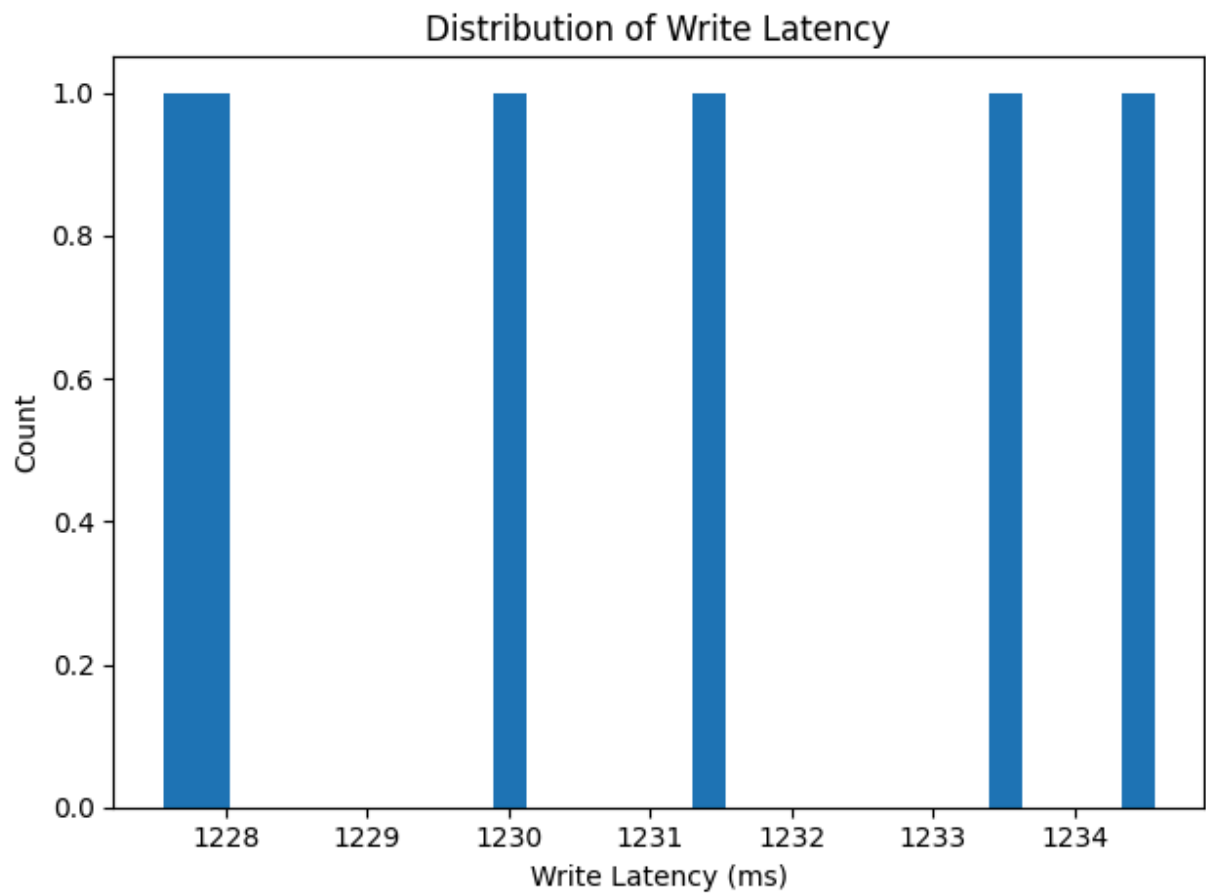
```
python load_tester/load_test.py --mode lf --strategy w5r1 --ops 500 --read_pct 99 --write_pct 1  
--outdir results/lf_w5r1_99r_1w
```

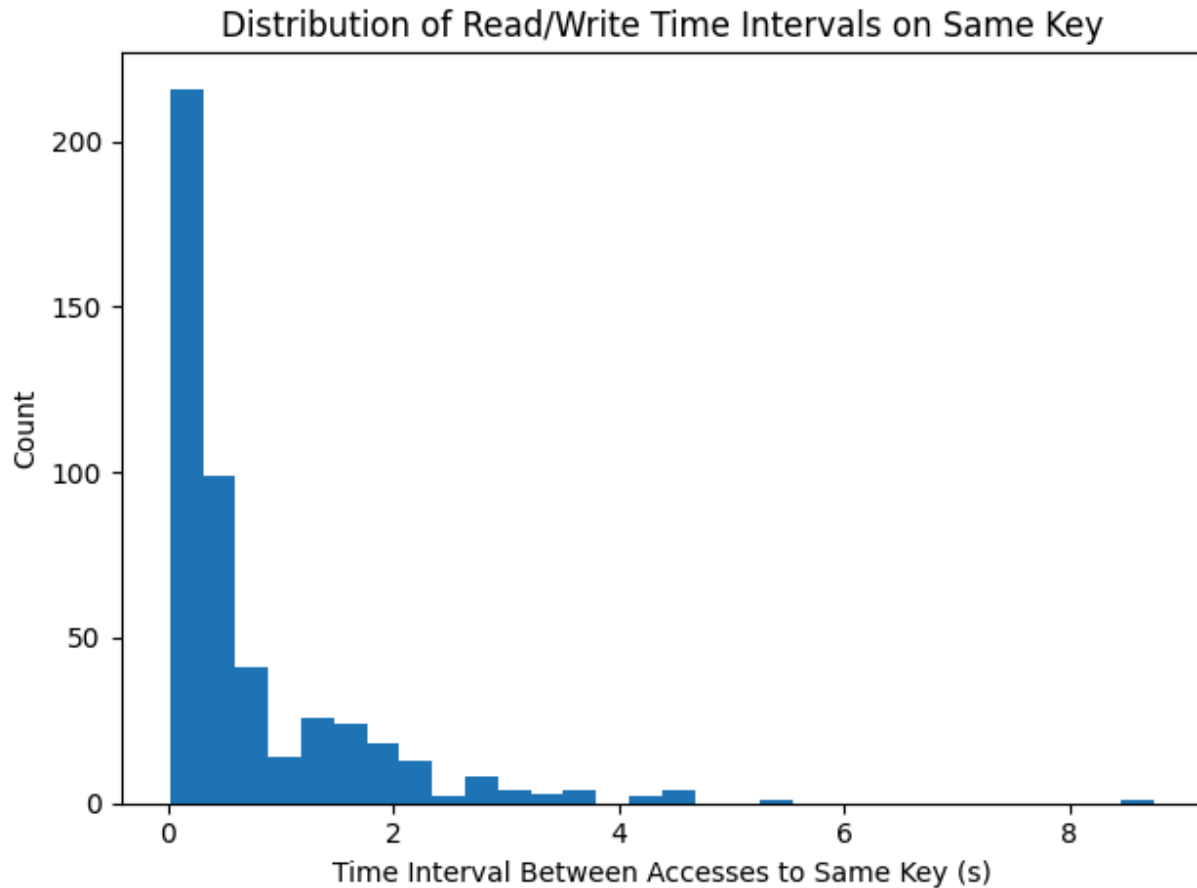
```
python load_tester/plot_results.py --indir results/lf_w5r1_99r_1w
```

HW10 > distributed-kv > results > lf_w5r1_99r_1w > {} summary.json > ...

```
1 {  
2   "mode": "lf",  
3   "strategy": "w5r1",  
4   "ops": 500,  
5   "read_pct": 99,  
6   "write_pct": 1,  
7   "stale_reads": 0,  
8   "total_reads": 494,  
9   "total_writes": 6,  
10  "avg_read_latency_ms": 13.49923481822093,  
11  "avg_write_latency_ms": 1230.812316663408  
12 }
```







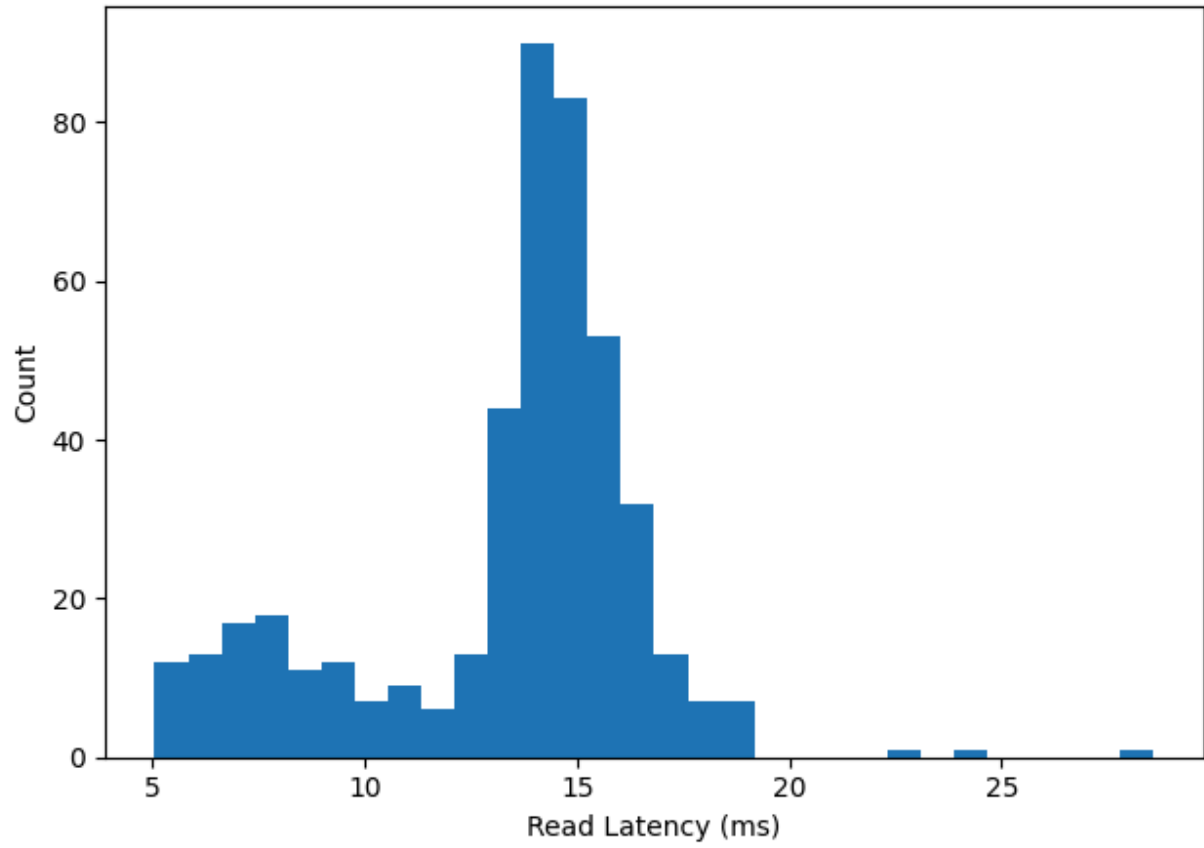
```
python load_tester/load_test.py --mode lf --strategy w5r1 --ops 500 --read_pct 90 --write_pct 10  
--outdir results/lf_w5r1_90r_10w
```

```
python load_tester/plot_results.py --indir results/lf_w5r1_90r_10w
```

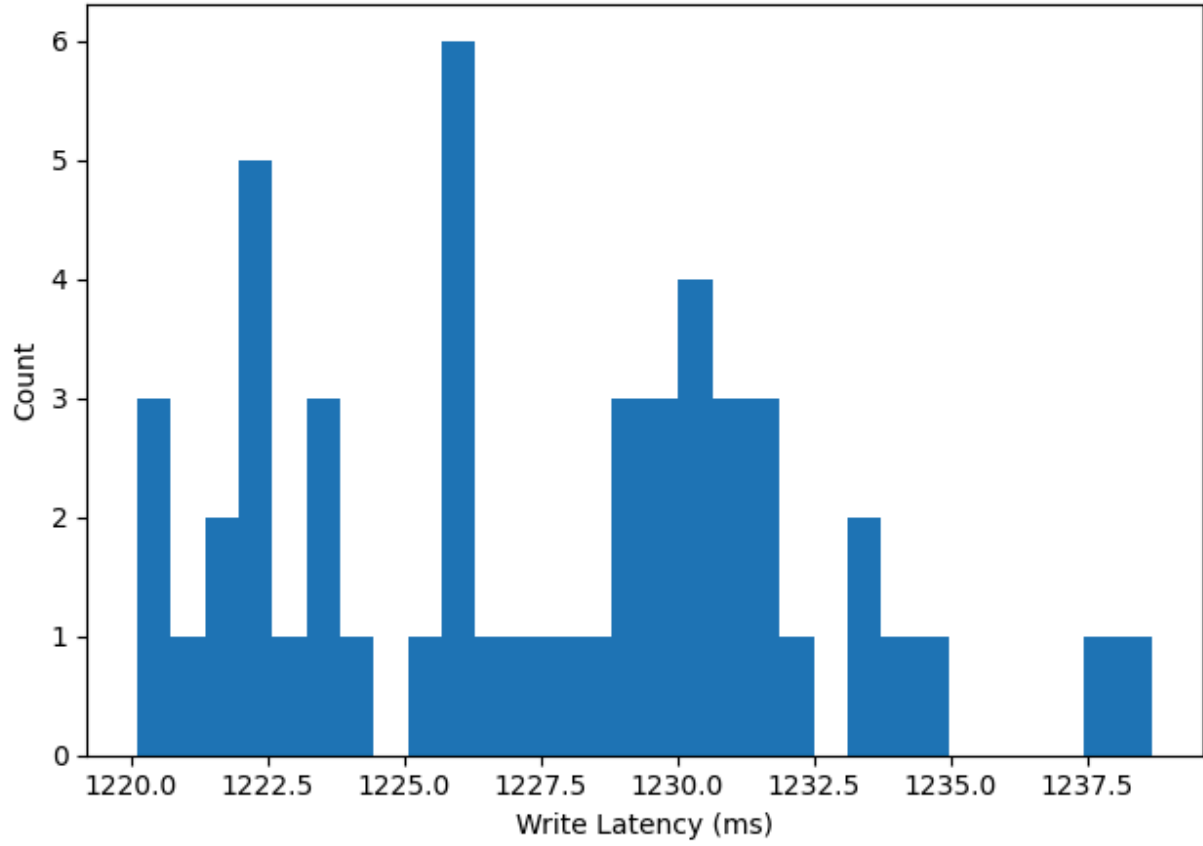
HW10 > distributed-kv > results > lf_w5r1_90r_10w > {} summary.json > ...

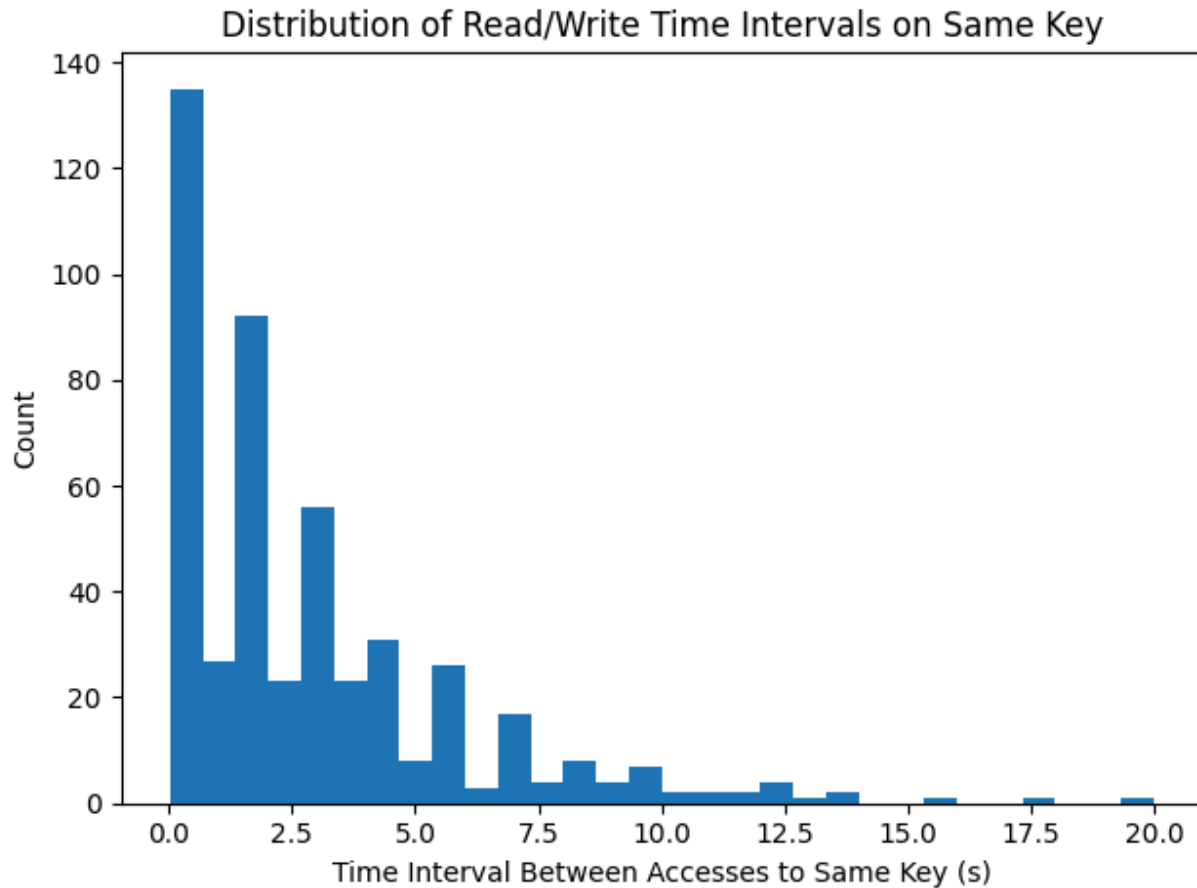
```
1 {  
2   "mode": "lf",  
3   "strategy": "w5r1",  
4   "ops": 500,  
5   "read_pct": 90,  
6   "write_pct": 10,  
7   "stale_reads": 0,  
8   "total_reads": 450,  
9   "total_writes": 50,  
10  "avg_read_latency_ms": 13.371991556163671,  
11  "avg_write_latency_ms": 1227.4676959984936  
12 }
```

Distribution of Read Latency



Distribution of Write Latency



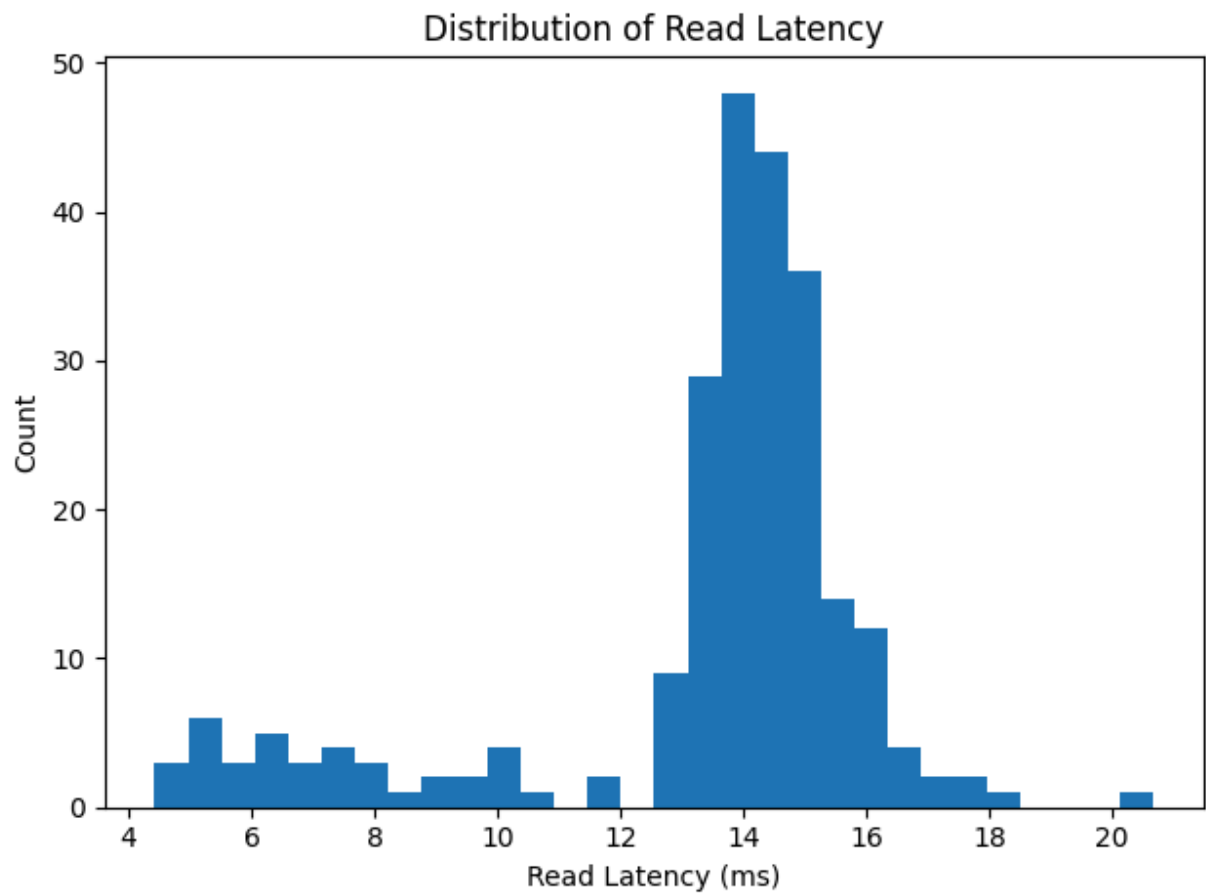


```
python load_tester/load_test.py --mode lf --strategy w5r1 --ops 500 --read_pct 50 --write_pct 50  
--outdir results/lf_w5r1_50r_50w
```

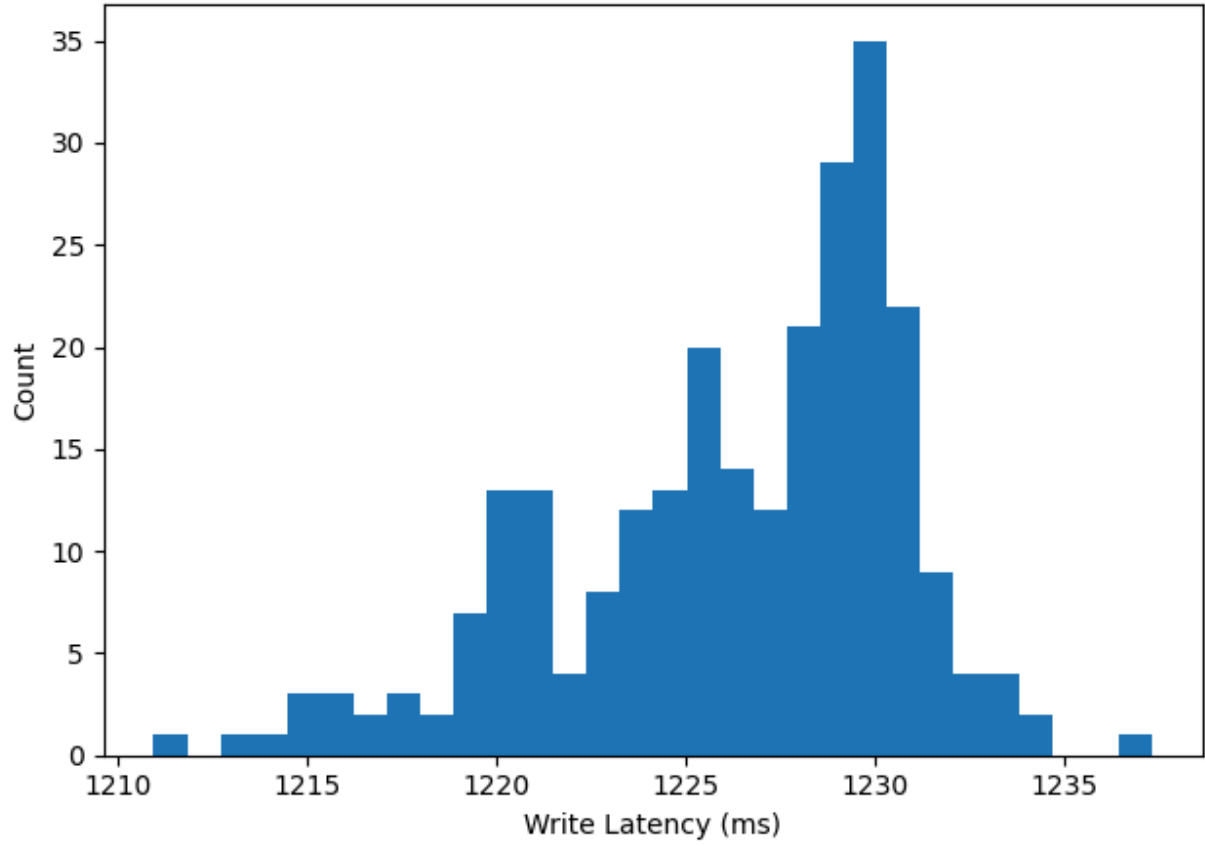
```
python load_tester/plot_results.py --indir results/lf_w5r1_50r_50w
```

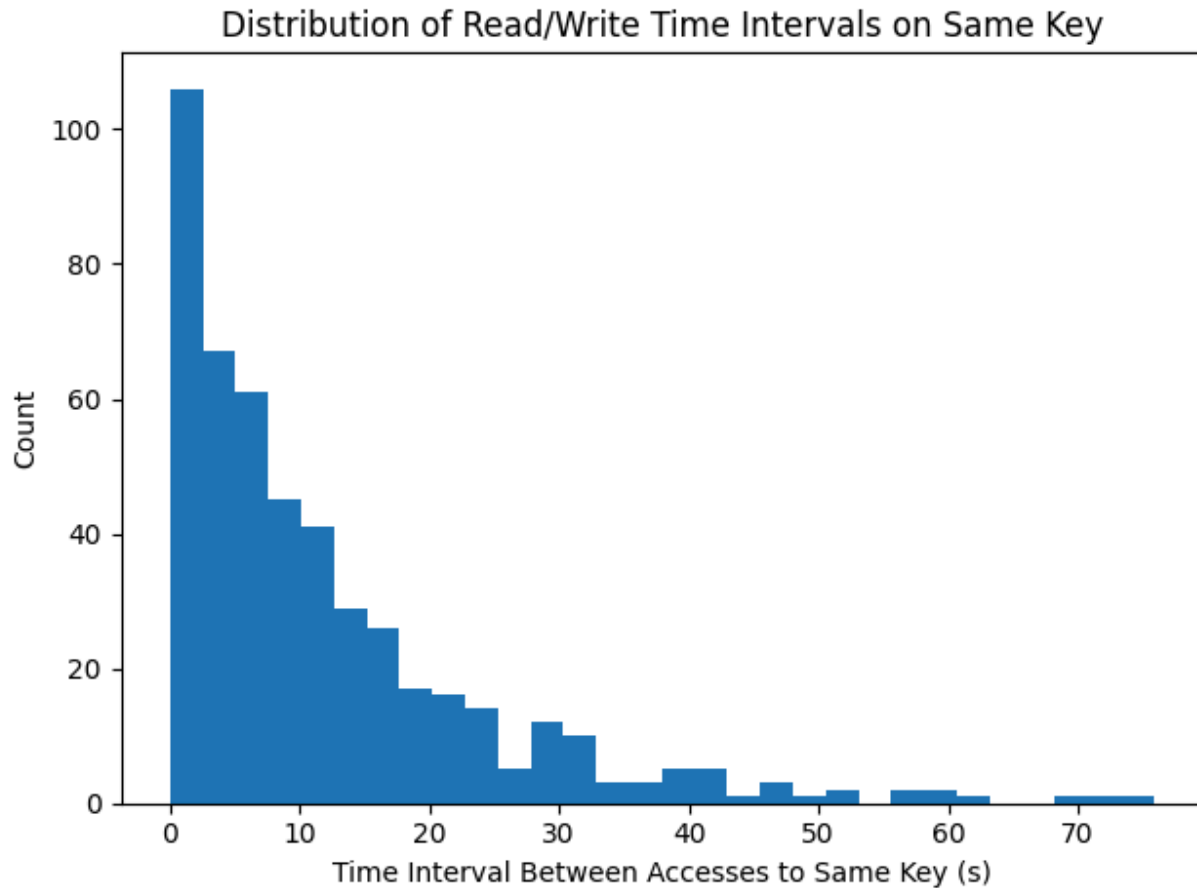
HW10 > distributed-kv > results > lf_w5r1_50r_50w > {} summary.json > ...

```
1 {  
2   "mode": "lf",  
3   "strategy": "w5r1",  
4   "ops": 500,  
5   "read_pct": 50,  
6   "write_pct": 50,  
7   "stale_reads": 0,  
8   "total_reads": 241,  
9   "total_writes": 259,  
10  "avg_read_latency_ms": 13.380088380883791,  
11  "avg_write_latency_ms": 1226.3294613916107  
12 }
```



Distribution of Write Latency



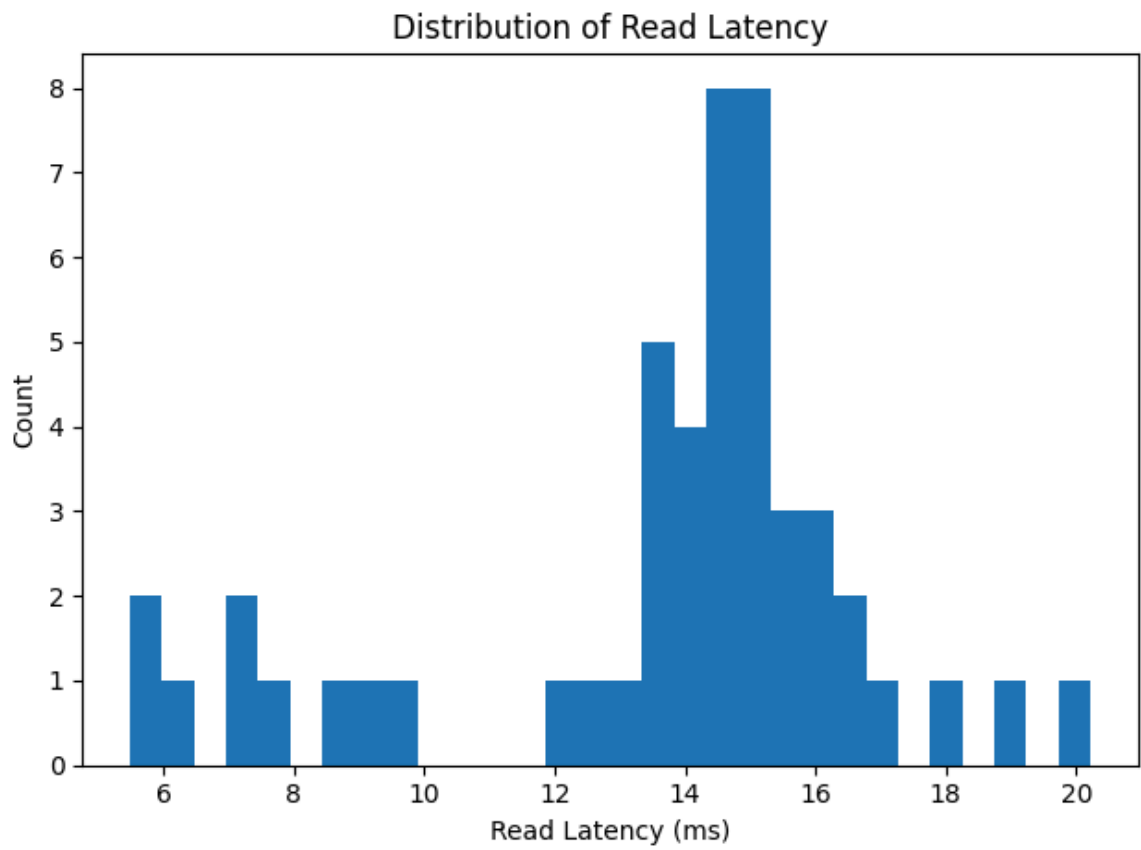


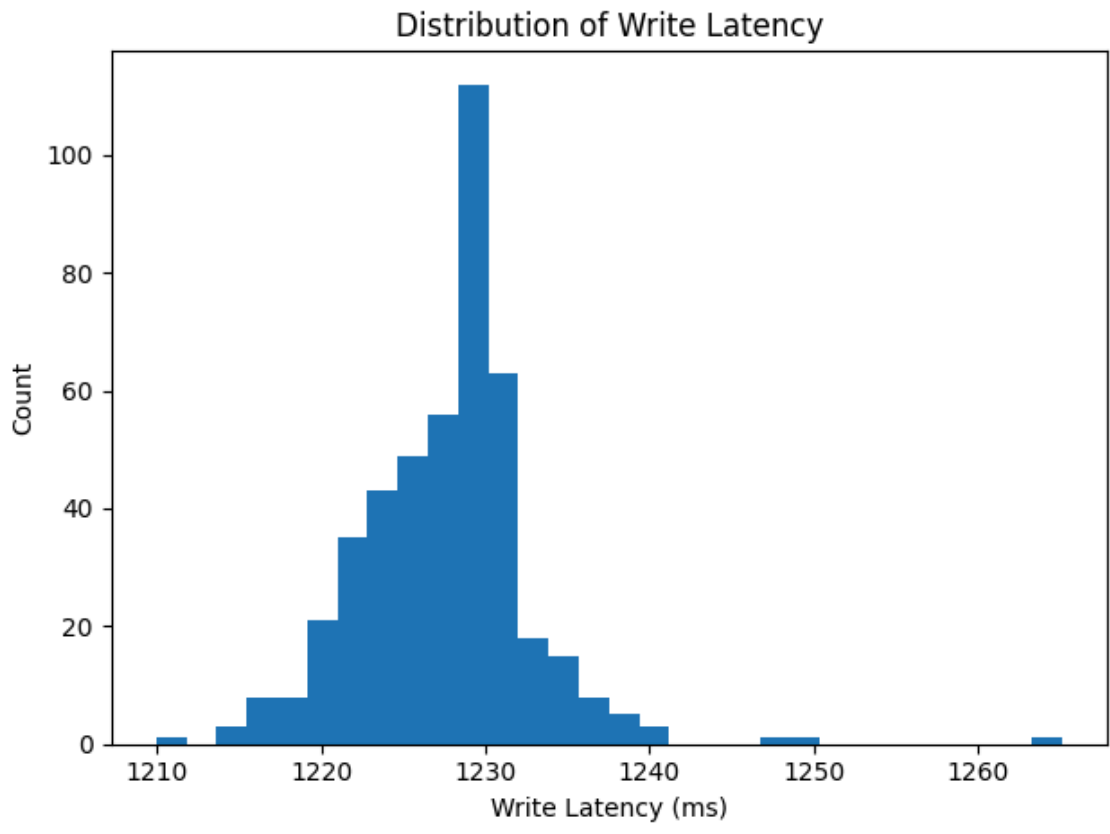
```
python load_tester/load_test.py --mode lf --strategy w5r1 --ops 500 --read_pct 10 --write_pct 90  
--outdir results/lf_w5r1_10r_90w
```

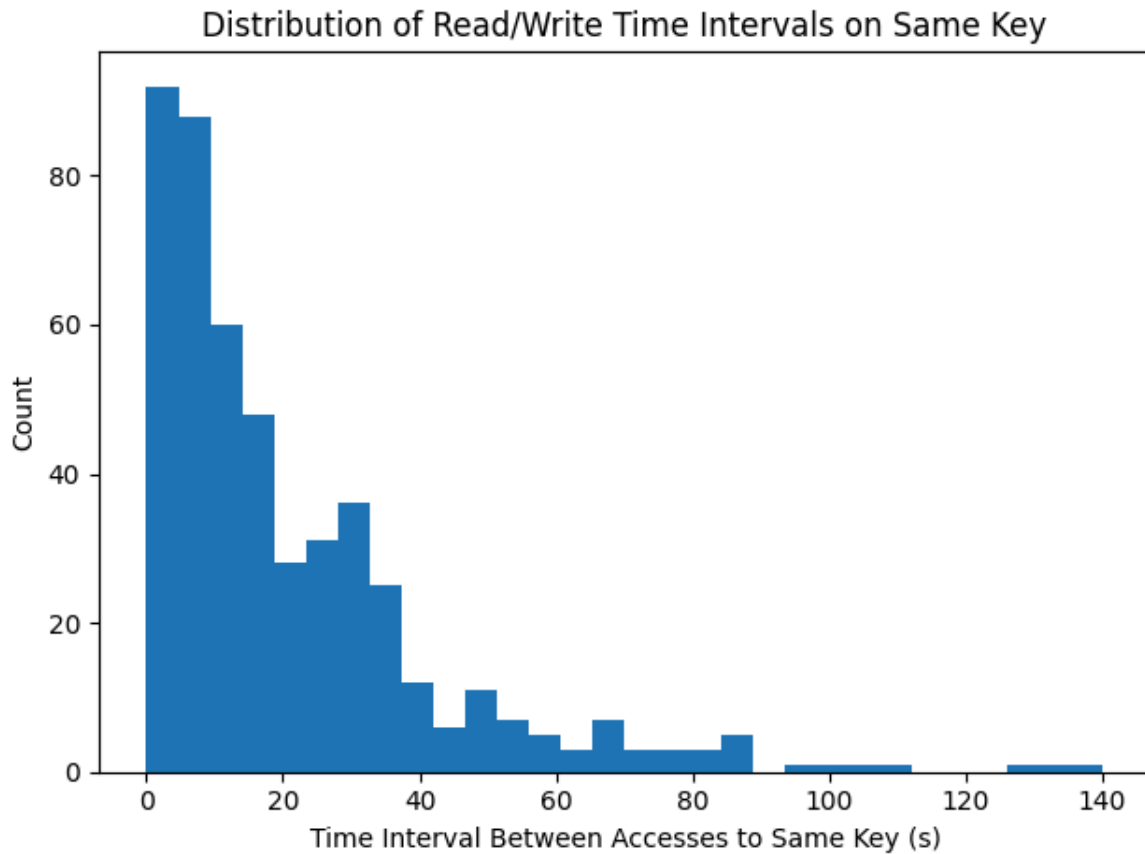
```
python load_tester/plot_results.py --indir results/lf_w5r1_10r_90w
```

HW10 > distributed-kv > results > lf_w5r1_10r_90w > {} summary.json > ...

```
1 {  
2   "mode": "lf",  
3   "strategy": "w5r1",  
4   "ops": 500,  
5   "read_pct": 10,  
6   "write_pct": 90,  
7   "stale_reads": 0,  
8   "total_reads": 49,  
9   "total_writes": 451,  
10  "avg_read_latency_ms": 13.603863267142478,  
11  "avg_write_latency_ms": 1227.531153435469  
12 }
```







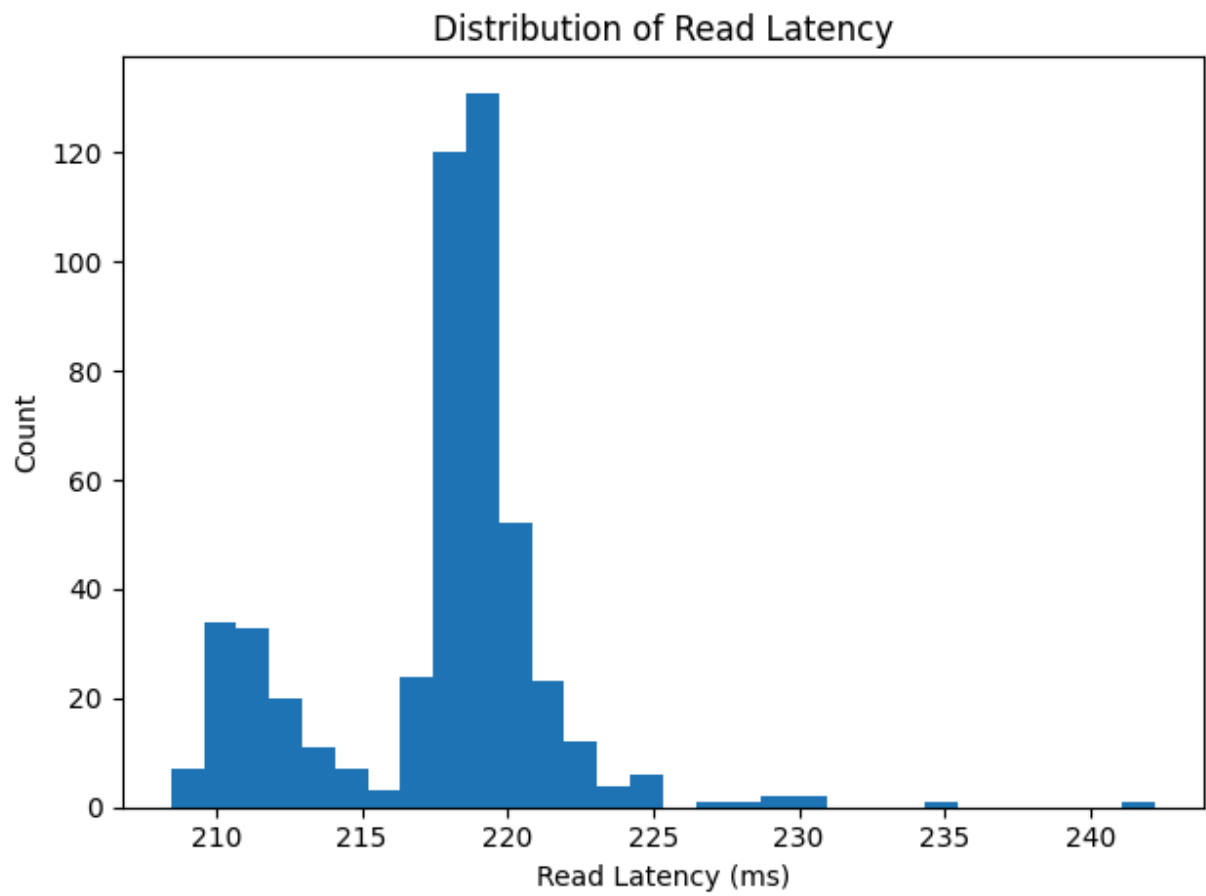
Run LF strategy W=1,R=5

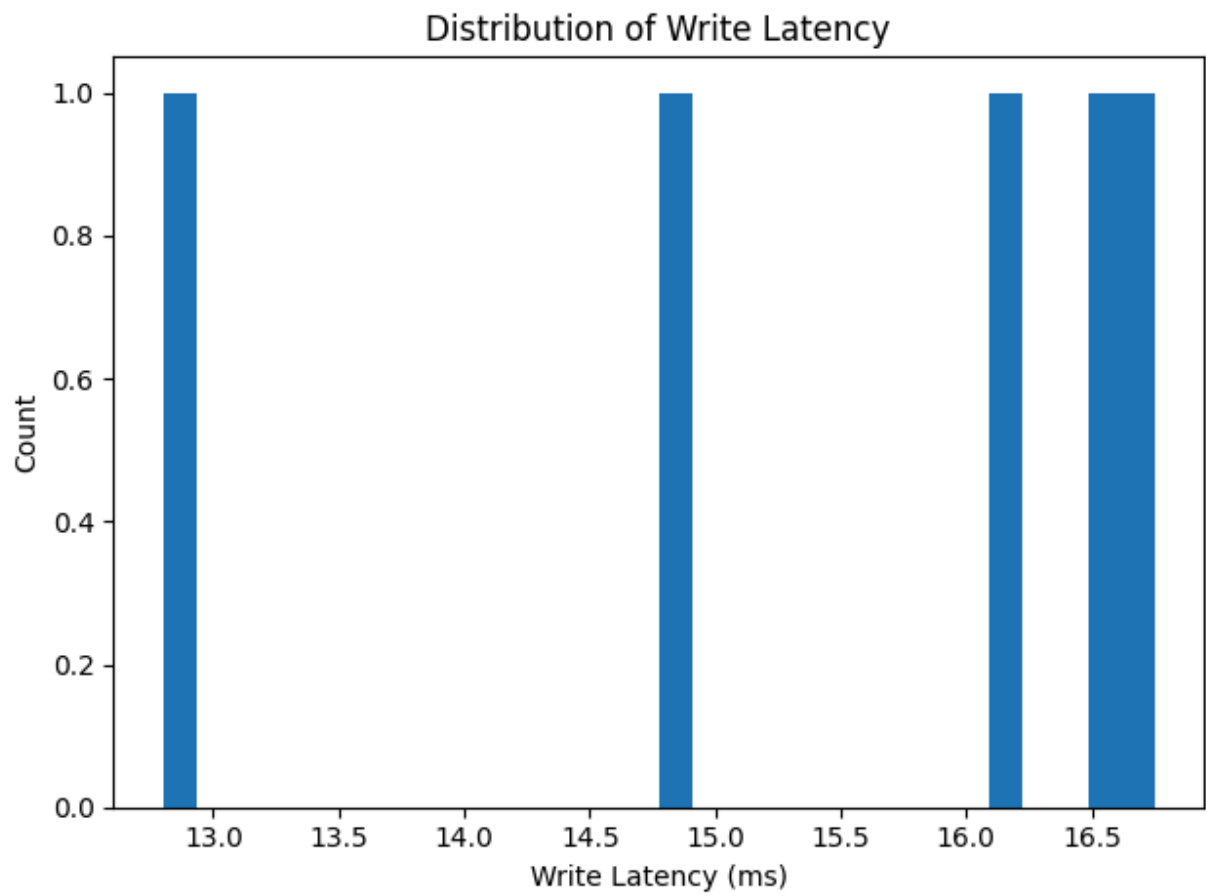
```
python load_tester/load_test.py --mode lf --strategy w1r5 --ops 500 --read_pct 99 --write_pct 1
--outdir results/lf_w1r5_99r_1w
```

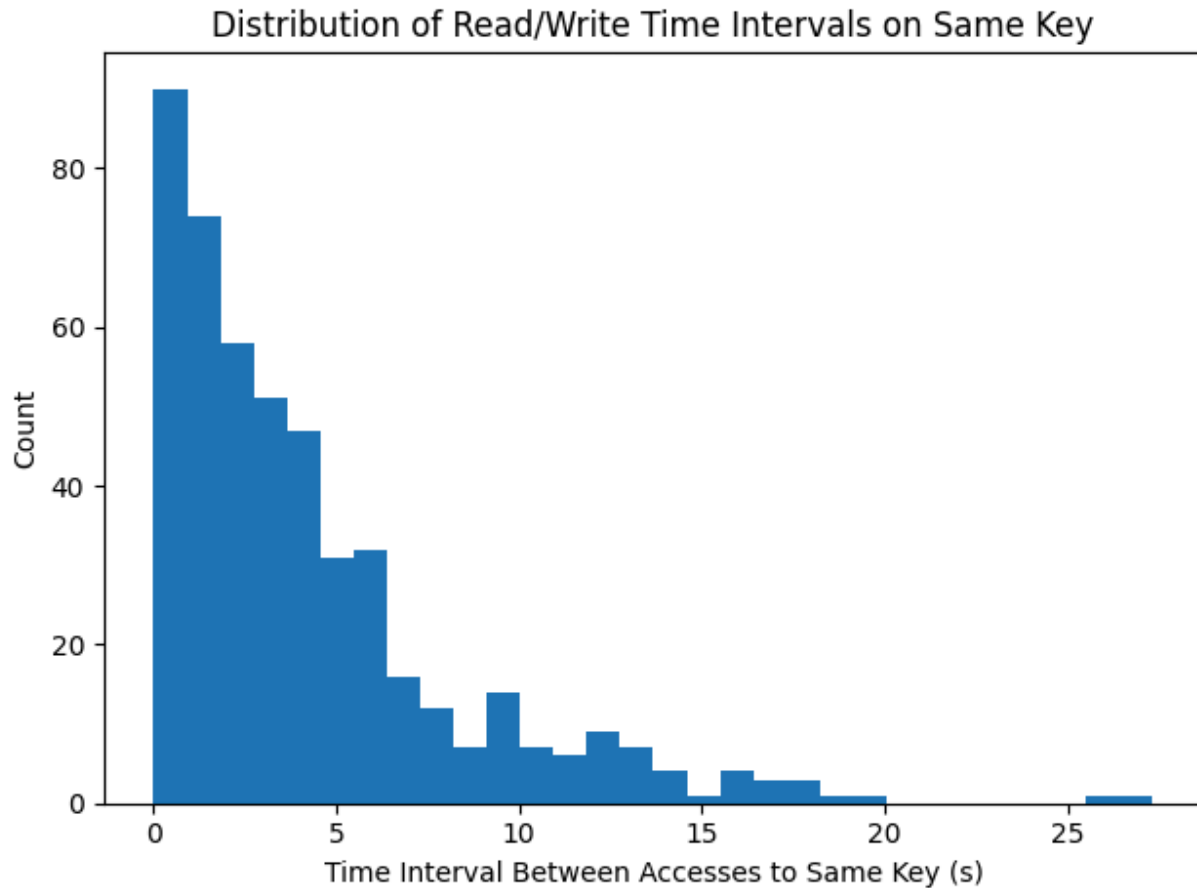
```
python load_tester/plot_results.py --indir results/lf_w1r5_99r_1w
```

HW10 > distributed-kv > results > lf_w1r5_99r_1w > {} summary.json > ...

```
1 {
2   "mode": "lf",
3   "strategy": "w1r5",
4   "ops": 500,
5   "read_pct": 99,
6   "write_pct": 1,
7   "stale_reads": 0,
8   "total_reads": 495,
9   "total_writes": 5,
10  "avg_read_latency_ms": 217.58028626197367,
11  "avg_write_latency_ms": 15.398480009753257
12 }
```







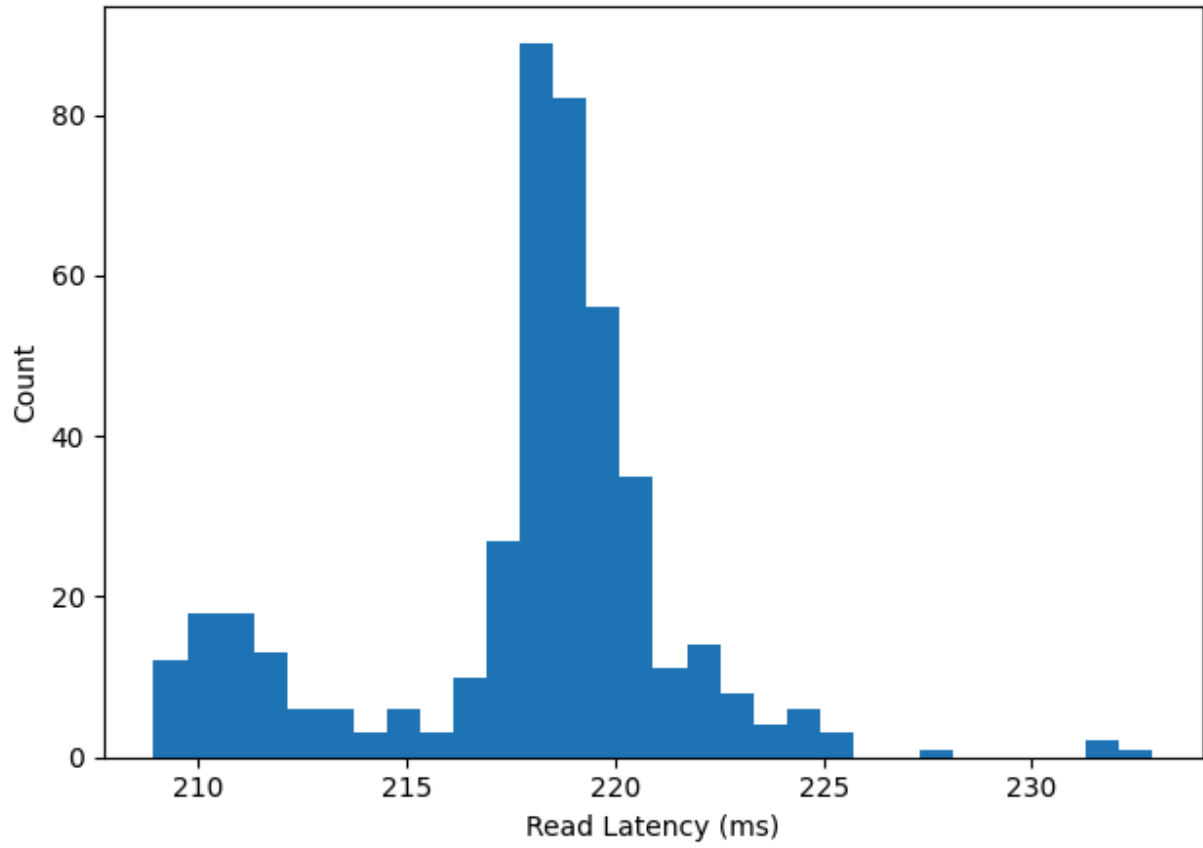
```
python load_tester/load_test.py --mode lf --strategy w1r5 --ops 500 --read_pct 90 --write_pct 10  
--outdir results/lf_w1r5_90r_10w
```

```
python load_tester/plot_results.py --indir results/lf_w1r5_90r_10w
```

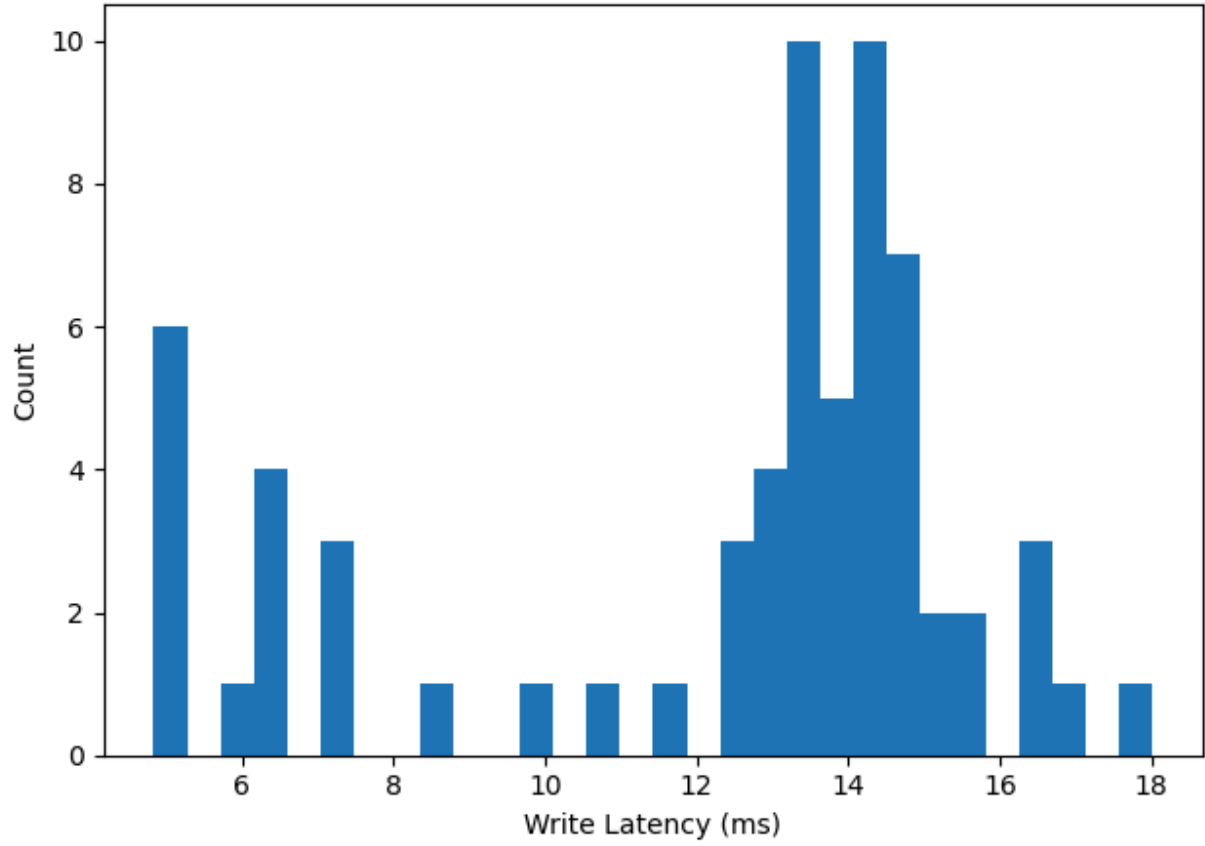
HW10 > distributed-kv > results > lf_w1r5_90r_10w > {} summary.json > ...

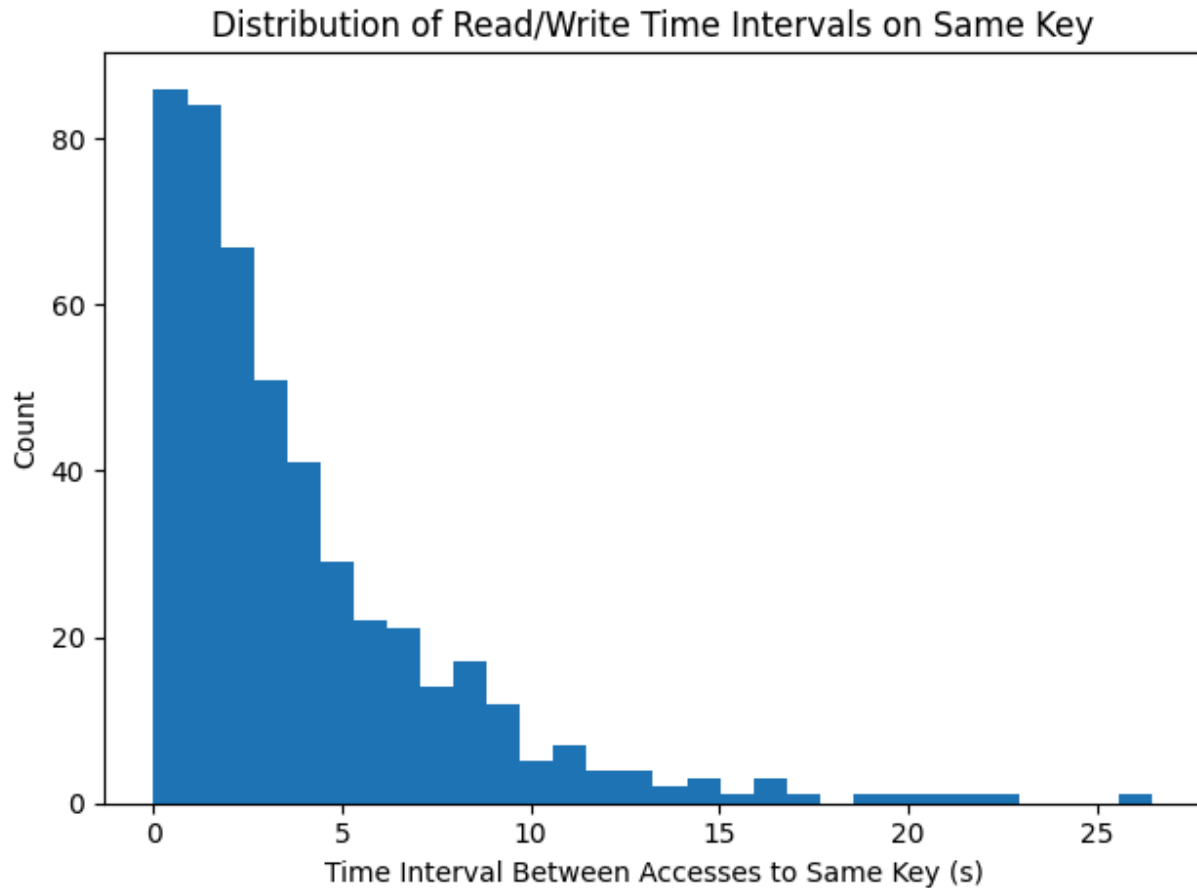
```
1 {  
2   "mode": "lf",  
3   "strategy": "w1r5",  
4   "ops": 500,  
5   "read_pct": 90,  
6   "write_pct": 10,  
7   "stale_reads": 0,  
8   "total_reads": 434,  
9   "total_writes": 66,  
10  "avg_read_latency_ms": 217.89290783468158,  
11  "avg_write_latency_ms": 12.24720757719214  
12 }
```


Distribution of Read Latency



Distribution of Write Latency





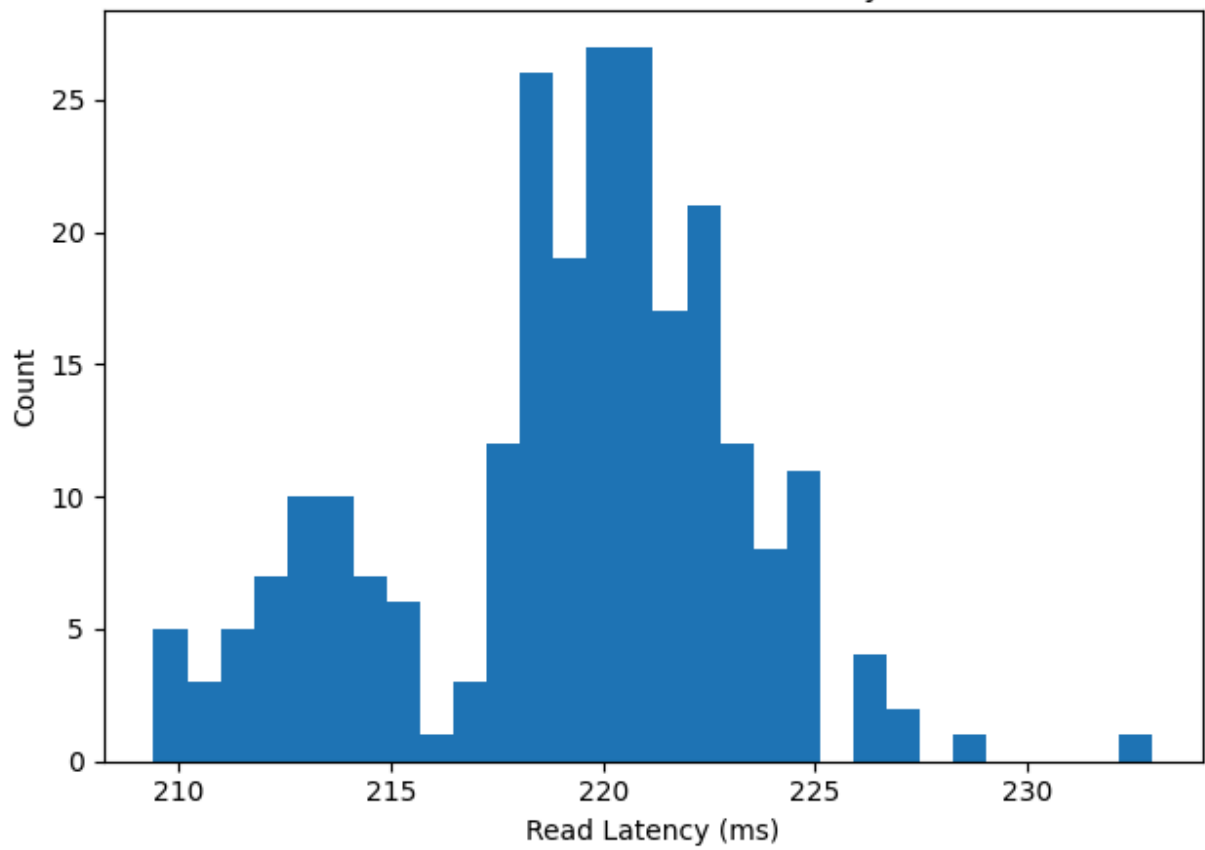
```
python load_tester/load_test.py --mode lf --strategy w1r5 --ops 500 --read_pct 50 --write_pct 50  
--outdir results/lf_w1r5_50r_50w
```

```
python load_tester/plot_results.py --indir results/lf_w1r5_50r_50w
```

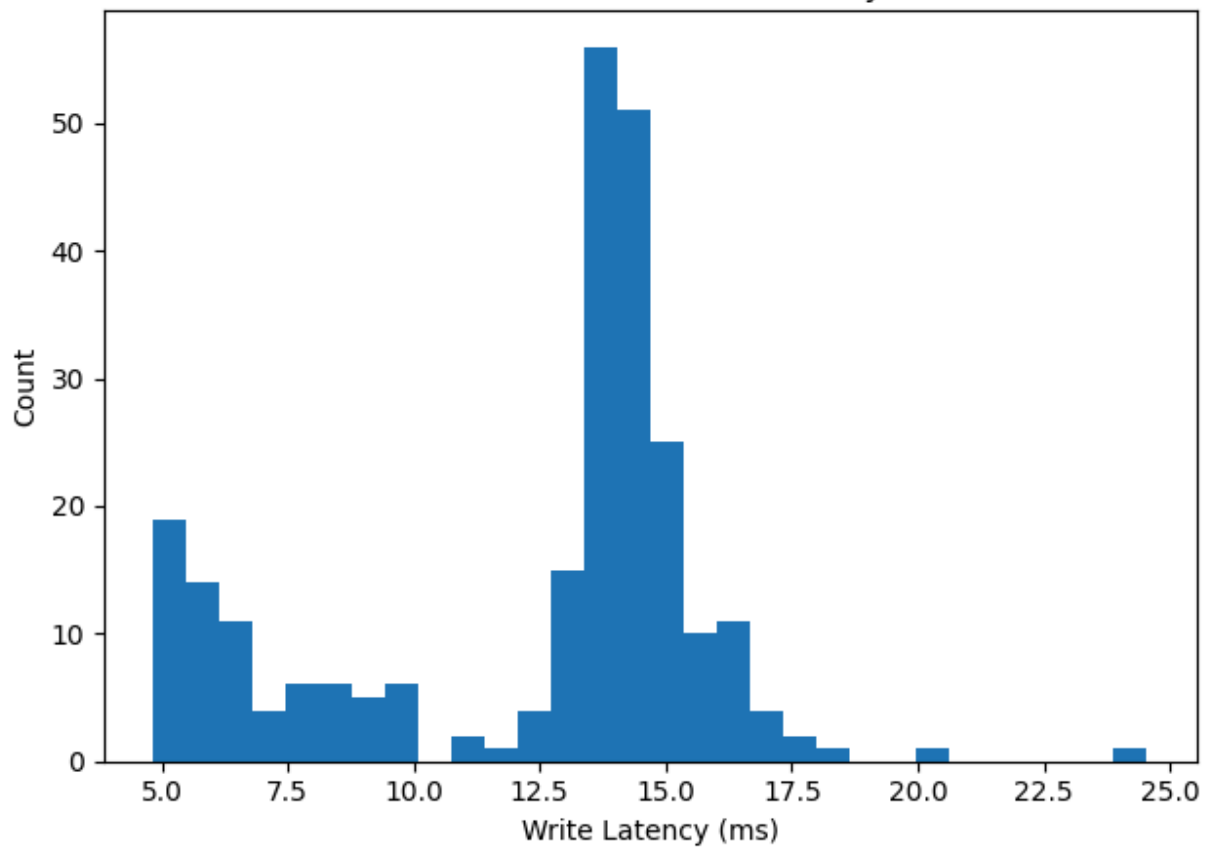
HW10 > distributed-kv > results > lf_w1r5_50r_50w > {} summary.json > ...

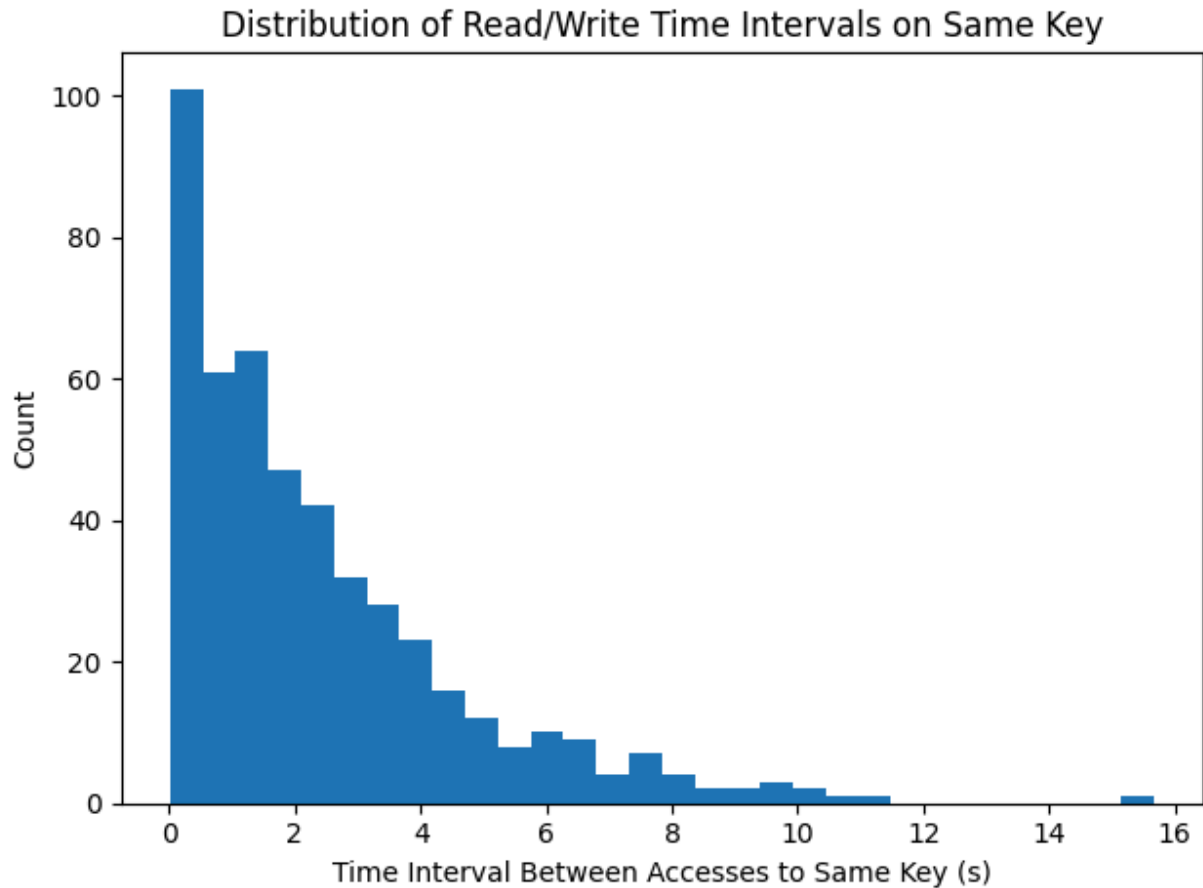
```
1 {  
2   "mode": "lf",  
3   "strategy": "w1r5",  
4   "ops": 500,  
5   "read_pct": 50,  
6   "write_pct": 50,  
7   "stale_reads": 0,  
8   "total_reads": 245,  
9   "total_writes": 255,  
10  "avg_read_latency_ms": 219.20863020314587,  
11  "avg_write_latency_ms": 12.310610587854741  
12 }
```

Distribution of Read Latency



Distribution of Write Latency





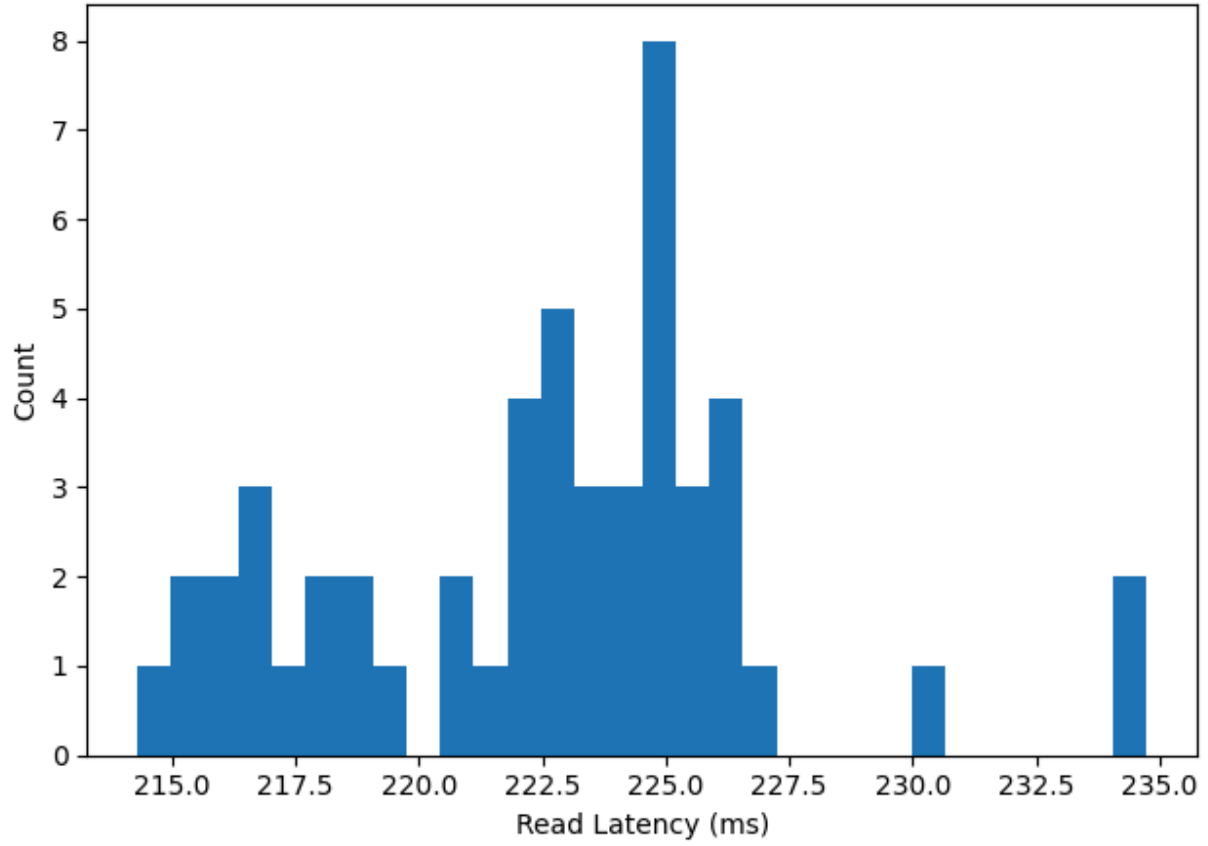
```
python load_tester/load_test.py --mode lf --strategy w1r5 --ops 500 --read_pct 10 --write_pct 90  
--outdir results/lf_w1r5_10r_90w
```

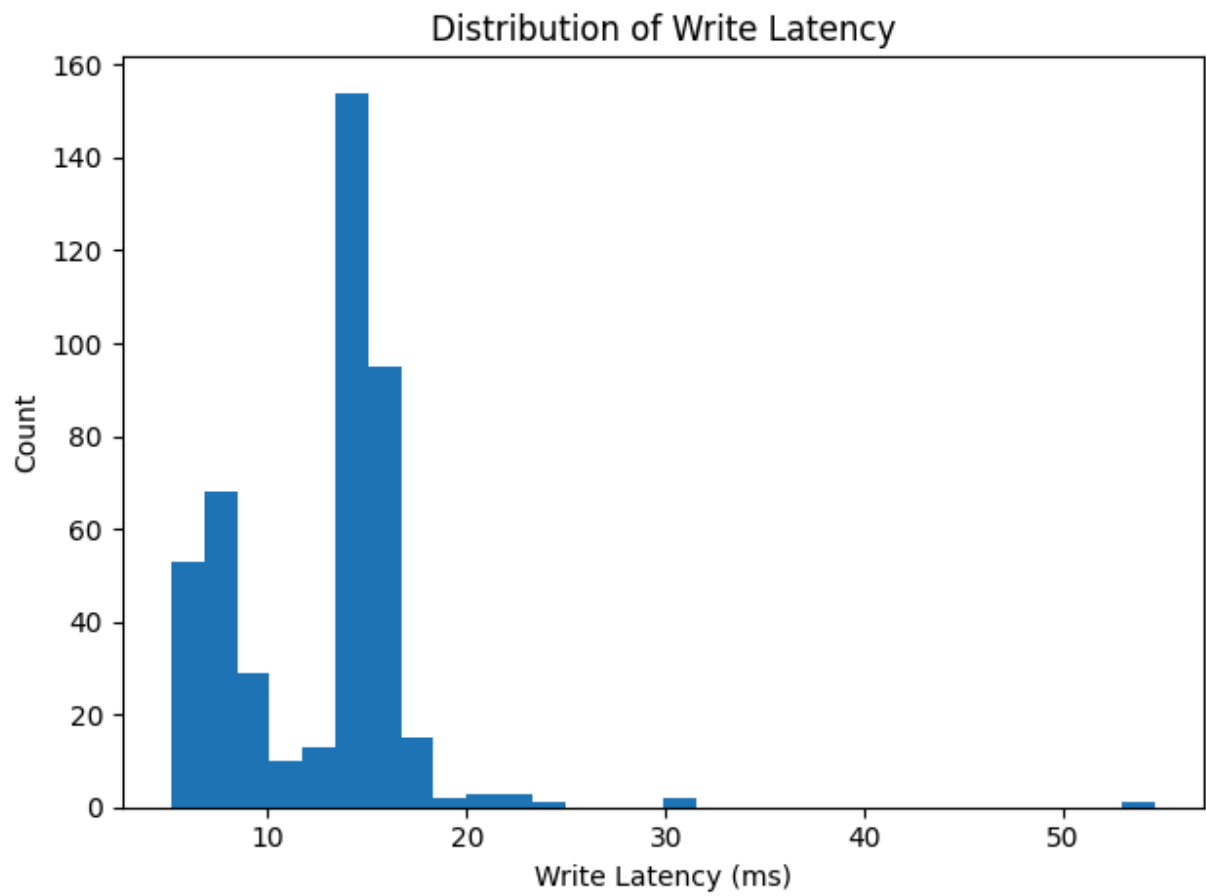
```
python load_tester/plot_results.py --indir results/lf_w1r5_10r_90w
```

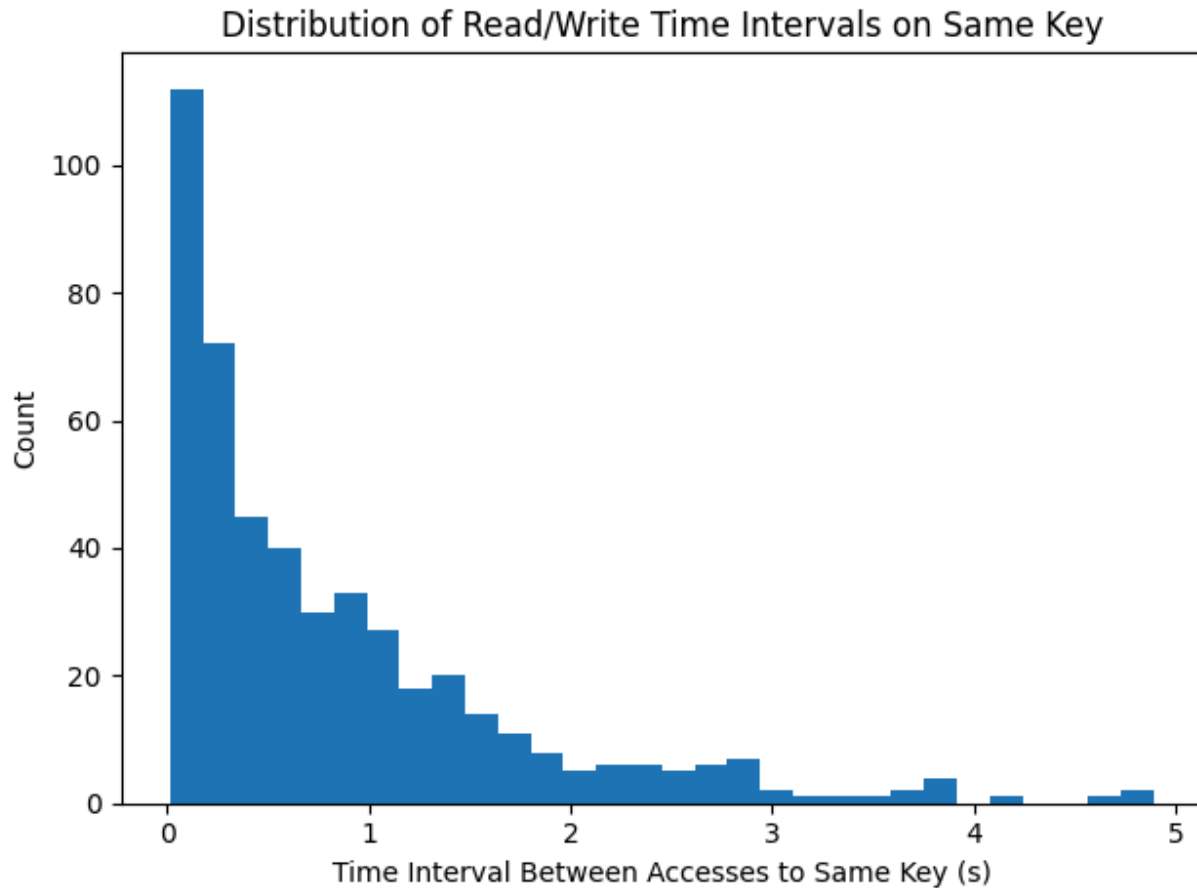
HW10 > distributed-kv > results > lf_w1r5_10r_90w > {} summary.json > ...

```
1 {  
2   "mode": "lf",  
3   "strategy": "w1r5",  
4   "ops": 500,  
5   "read_pct": 10,  
6   "write_pct": 90,  
7   "stale_reads": 0,  
8   "total_reads": 51,  
9   "total_writes": 449,  
10  "avg_read_latency_ms": 222.58188039082668,  
11  "avg_write_latency_ms": 12.558424499203225  
12 }
```

Distribution of Read Latency







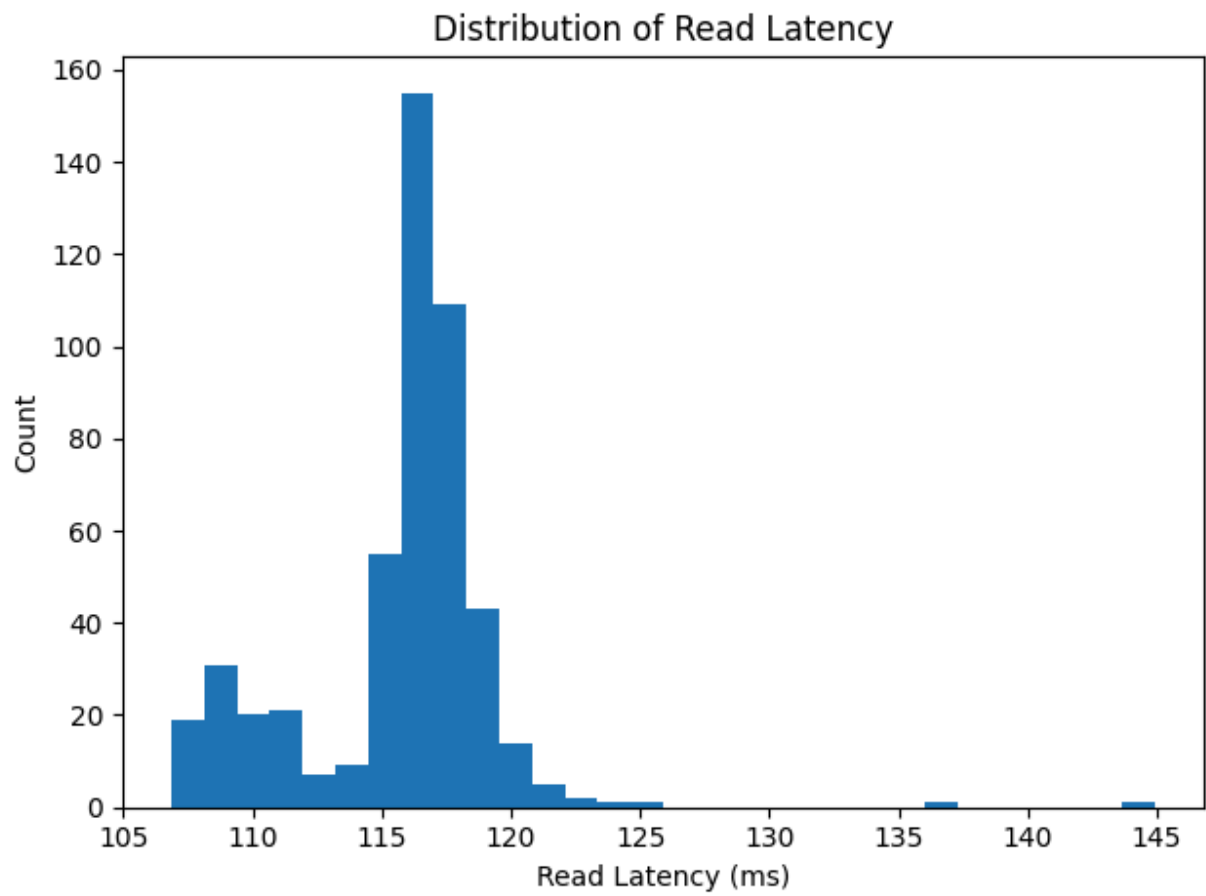
Run LF strategy W=3,R=3

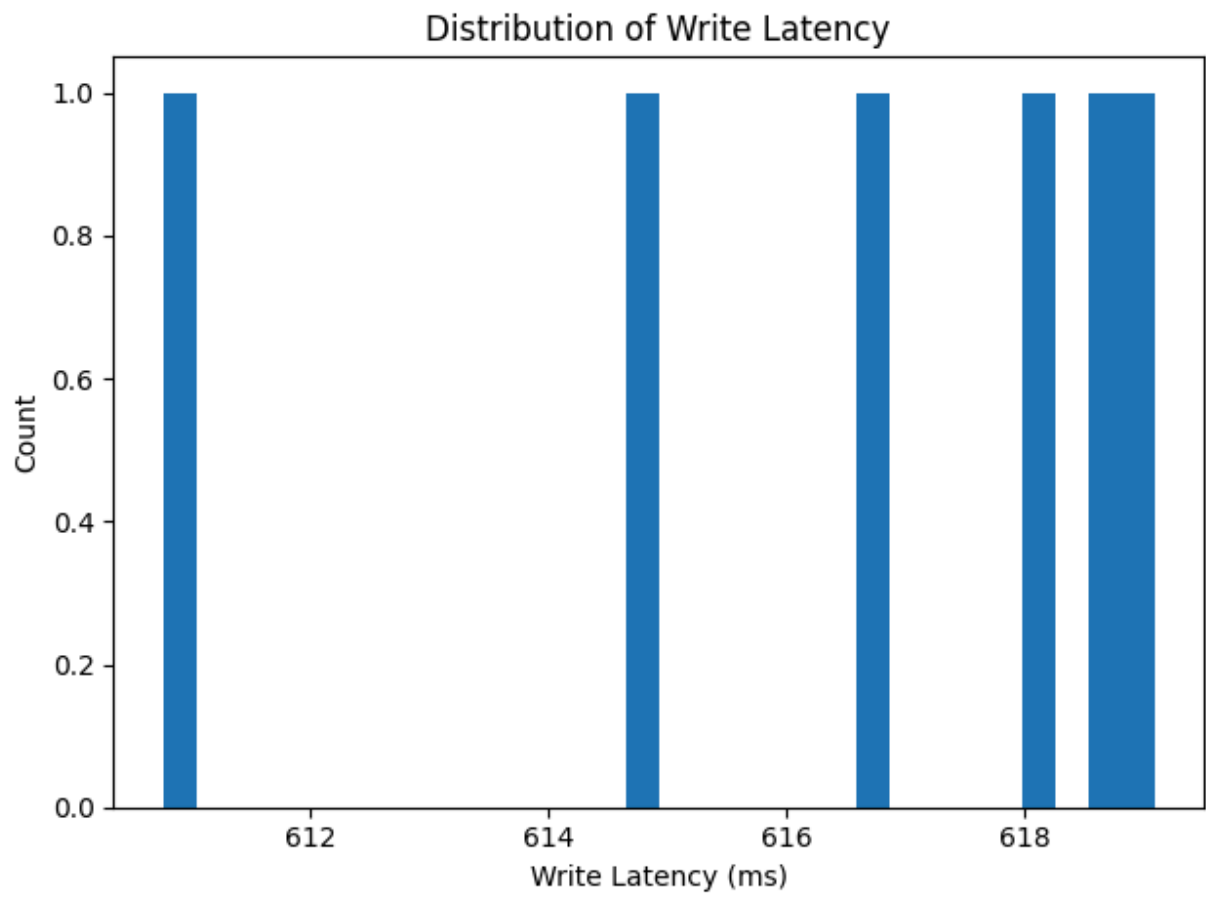
```
python load_tester/load_test.py --mode lf --strategy w3r3 --ops 500 --read_pct 99 --write_pct 1  
--outdir results/lf_w3r3_99r_1w
```

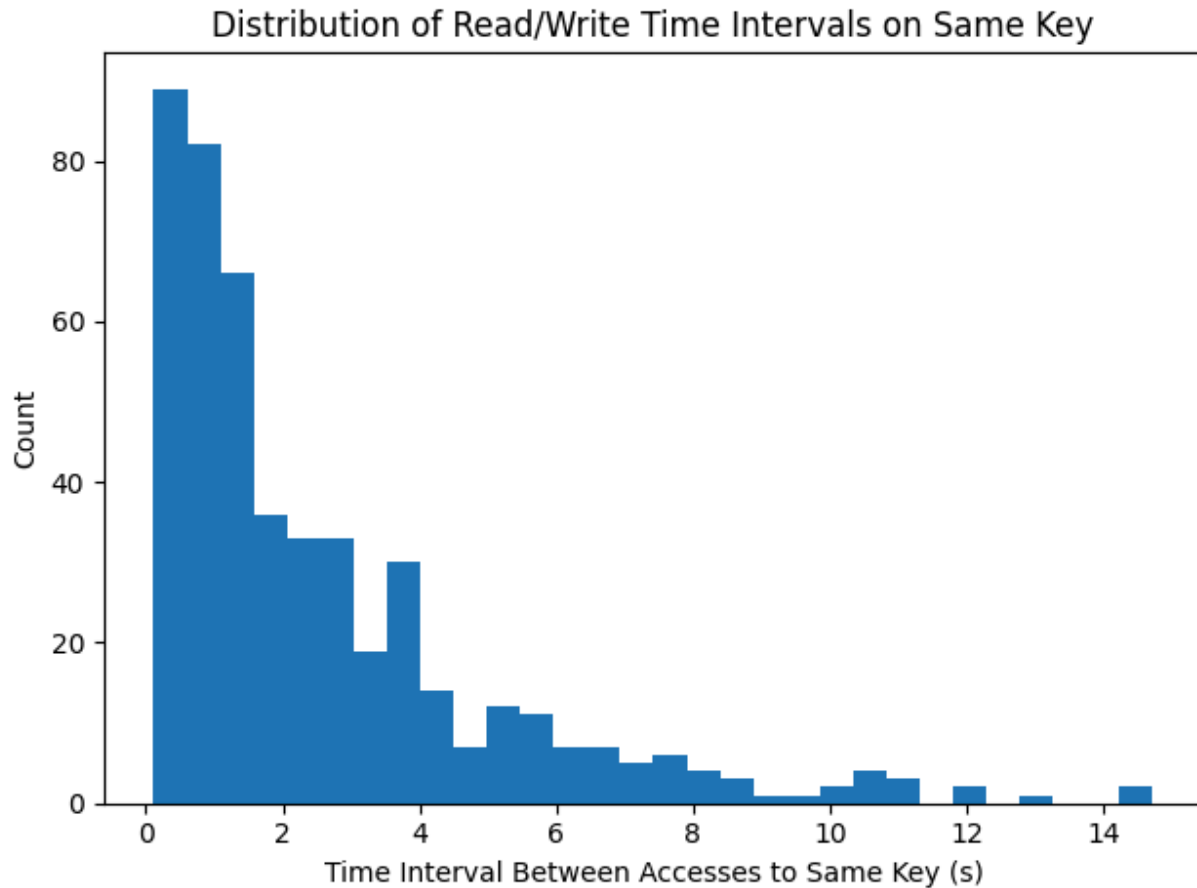
```
python load_tester/plot_results.py --indir results/lf_w3r3_99r_1w
```

HW10 > distributed-kv > results > lf_w3r3_99r_1w > summary.json > ...

```
1 {  
2   "mode": "lf",  
3   "strategy": "w3r3",  
4   "ops": 500,  
5   "read_pct": 99,  
6   "write_pct": 1,  
7   "stale_reads": 0,  
8   "total_reads": 494,  
9   "total_writes": 6,  
10  "avg_read_latency_ms": 115.62791538459196,  
11  "avg_write_latency_ms": 616.3850833351413  
12 }
```







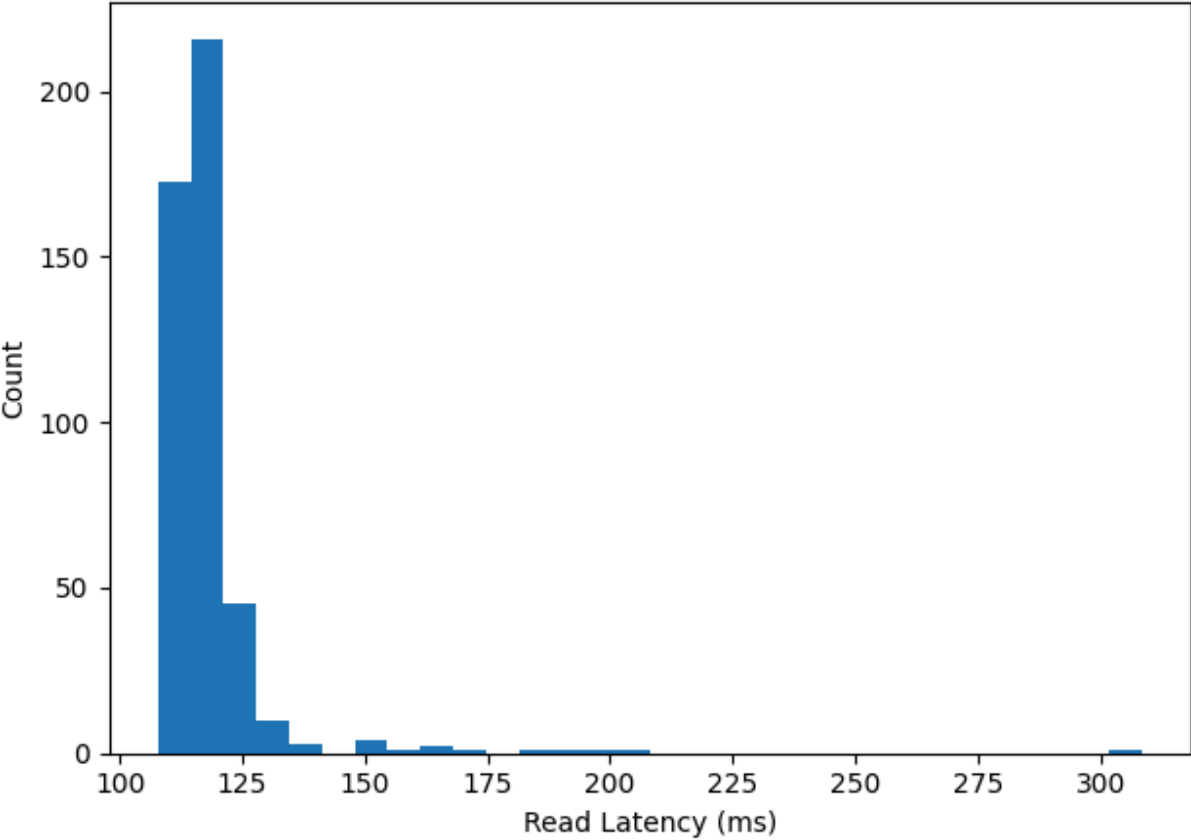
```
python load_tester/load_test.py --mode lf --strategy w3r3 --ops 500 --read_pct 90 --write_pct 10  
--outdir results/lf_w3r3_90r_10w
```

```
python load_tester/plot_results.py --indir results/lf_w3r3_90r_10w
```

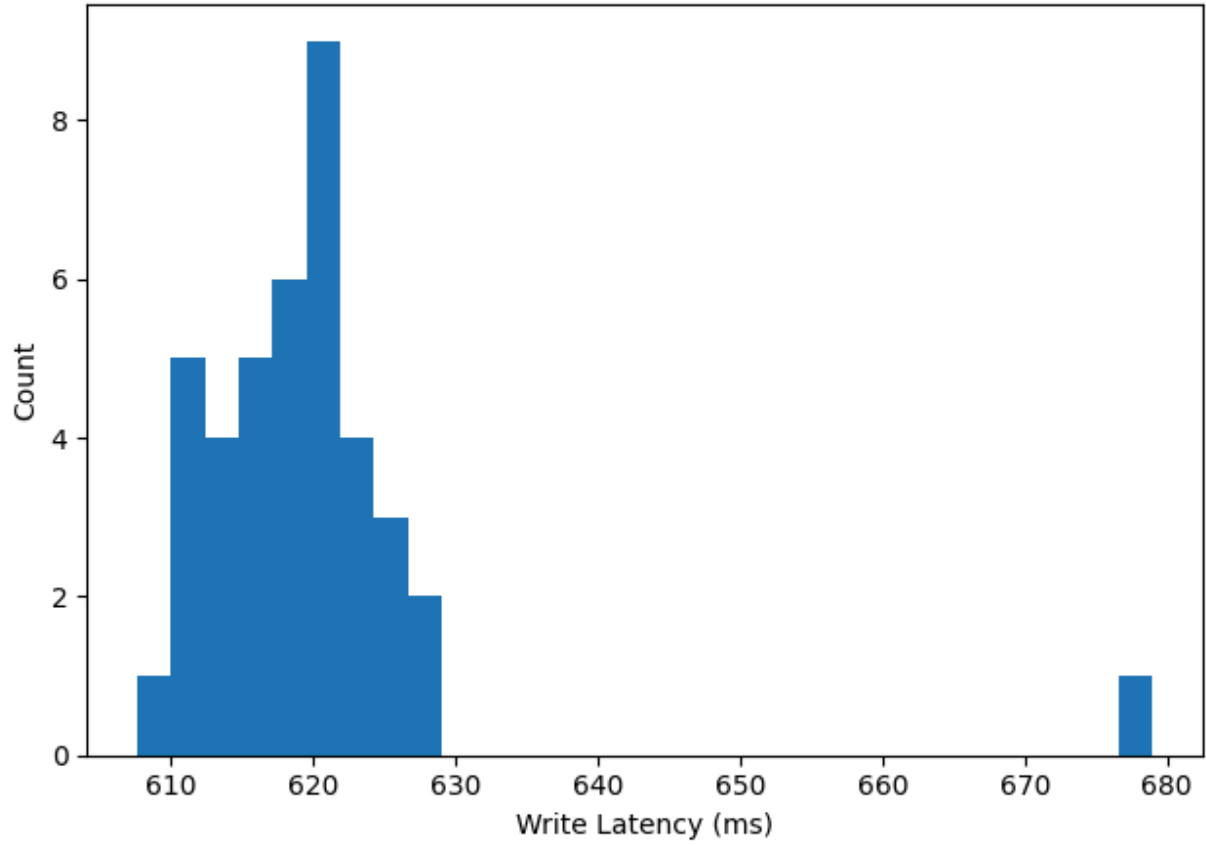
HW10 > distributed-kv > results > lf_w3r3_90r_10w > [summary.json](#) > ...

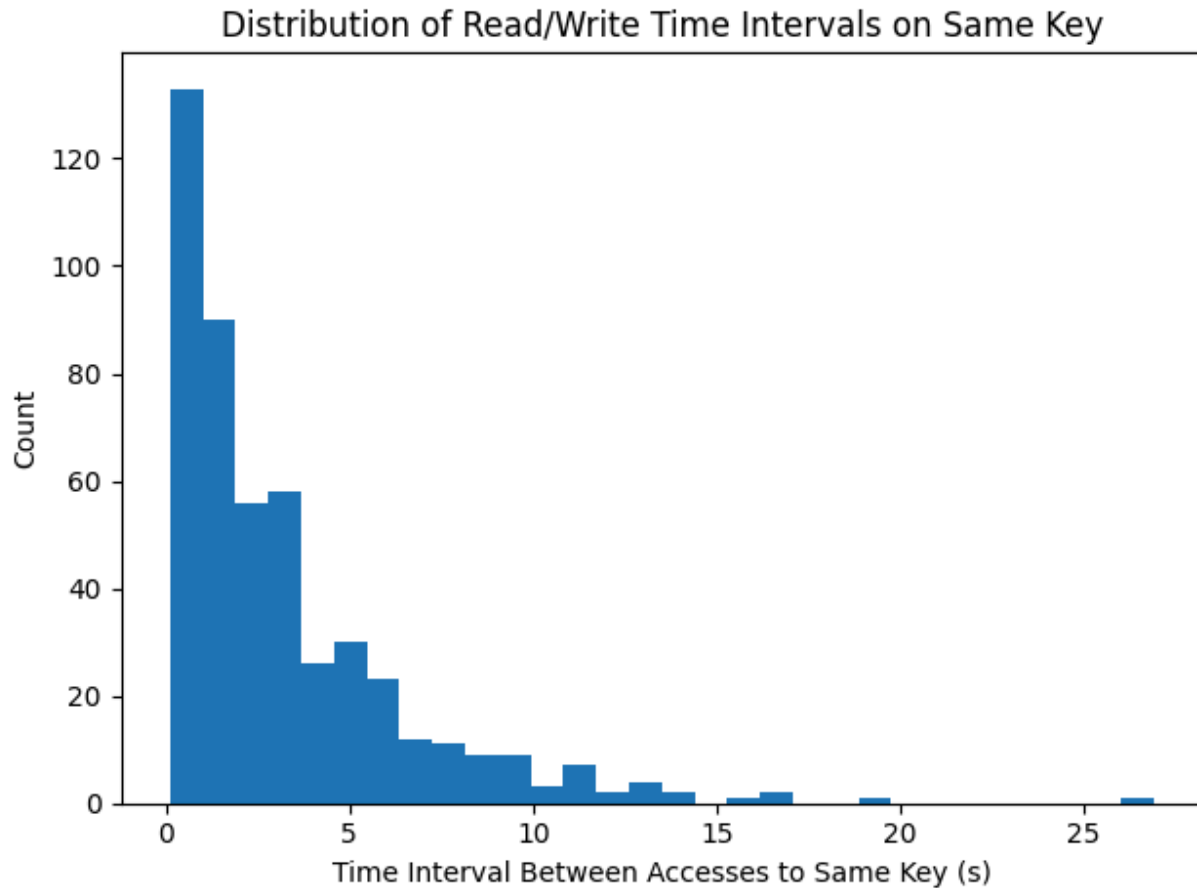
```
1 {  
2   "mode": "lf",  
3   "strategy": "w3r3",  
4   "ops": 500,  
5   "read_pct": 90,  
6   "write_pct": 10,  
7   "stale_reads": 0,  
8   "total_reads": 460,  
9   "total_writes": 40,  
10  "avg_read_latency_ms": 118.53854086935905,  
11  "avg_write_latency_ms": 619.8730424977839  
12 }
```

Distribution of Read Latency



Distribution of Write Latency





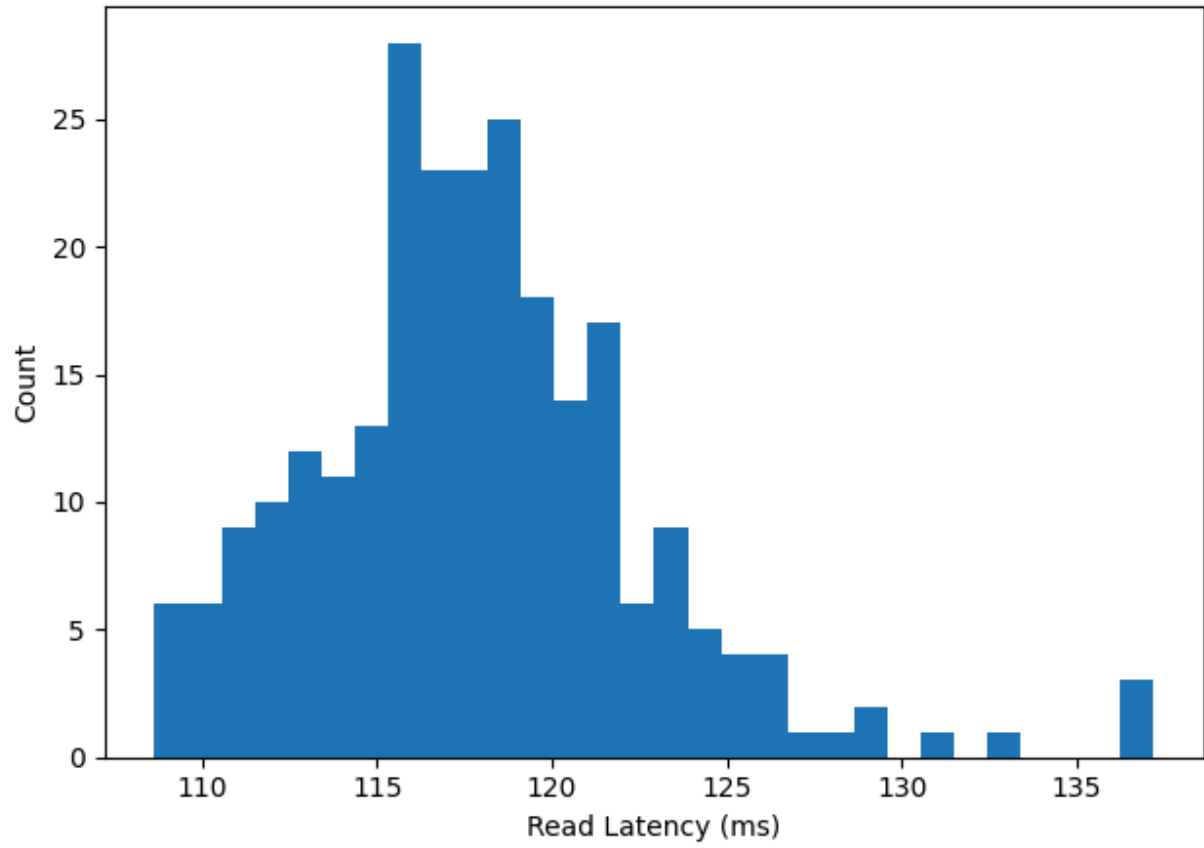
```
python load_tester/load_test.py --mode lf --strategy w3r3 --ops 500 --read_pct 50 --write_pct 50  
--outdir results/lf_w3r3_50r_50w
```

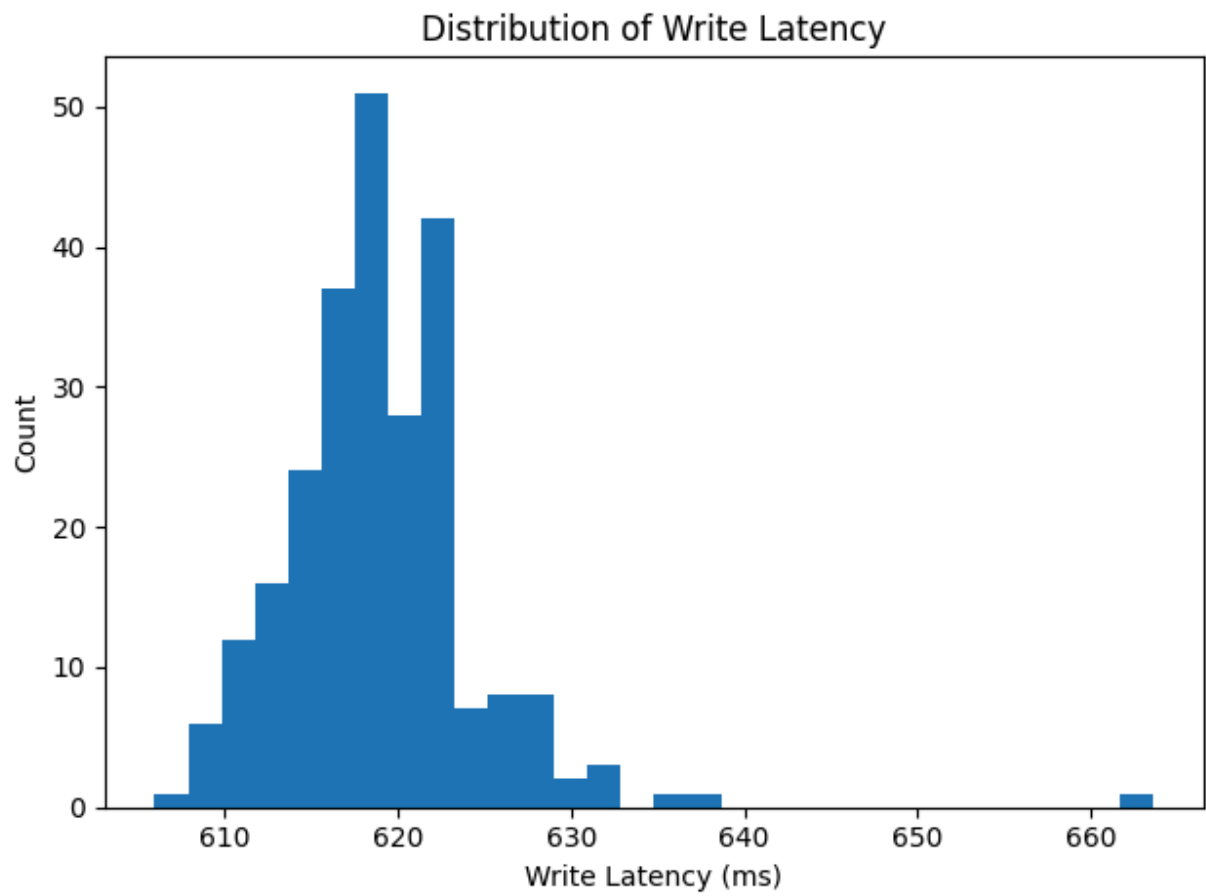
```
python load_tester/plot_results.py --indir results/lf_w3r3_50r_50w
```

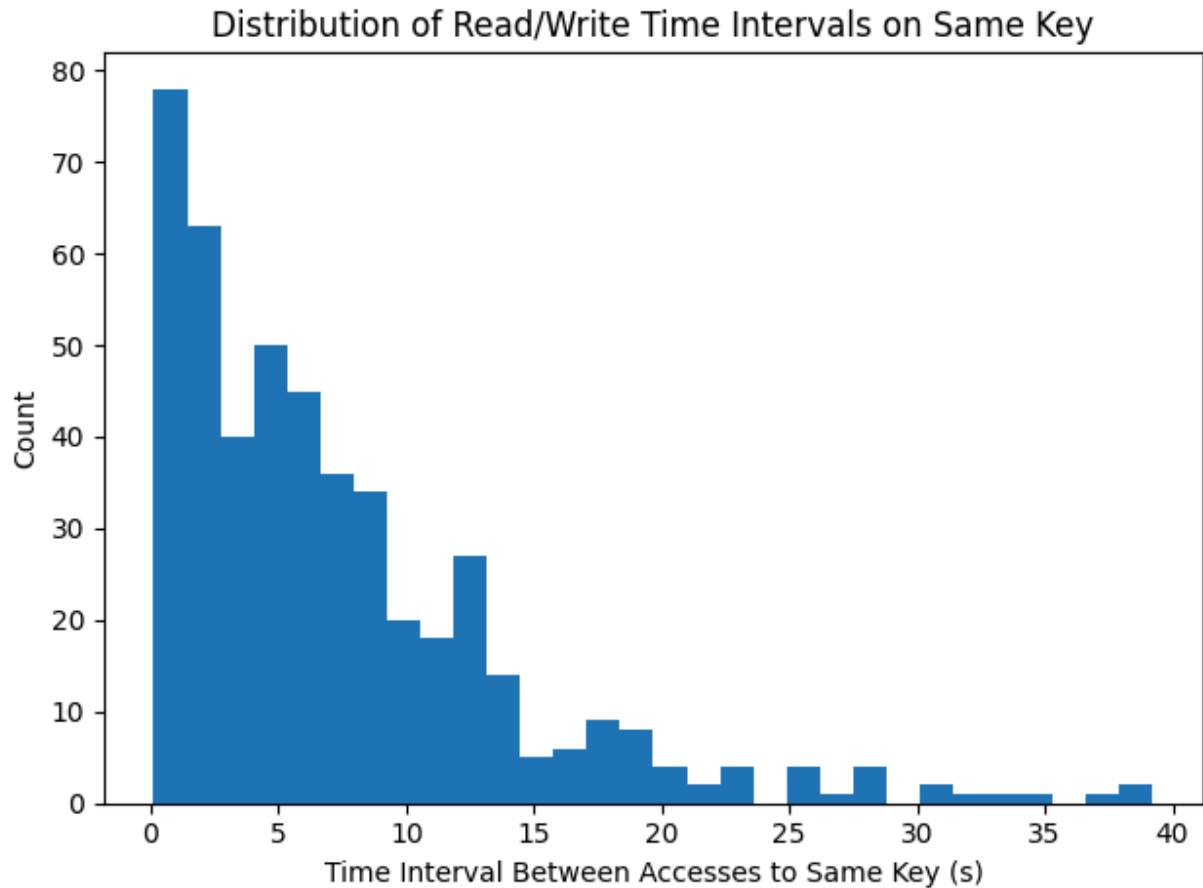
HW10 > distributed-kv > results > lf_w3r3_50r_50w > {} summary.json > ...

```
1 {  
2   "mode": "lf",  
3   "strategy": "w3r3",  
4   "ops": 500,  
5   "read_pct": 50,  
6   "write_pct": 50,  
7   "stale_reads": 0,  
8   "total_reads": 252,  
9   "total_writes": 248,  
10  "avg_read_latency_ms": 117.84774920670316,  
11  "avg_write_latency_ms": 618.9551092757029  
12 }
```

Distribution of Read Latency





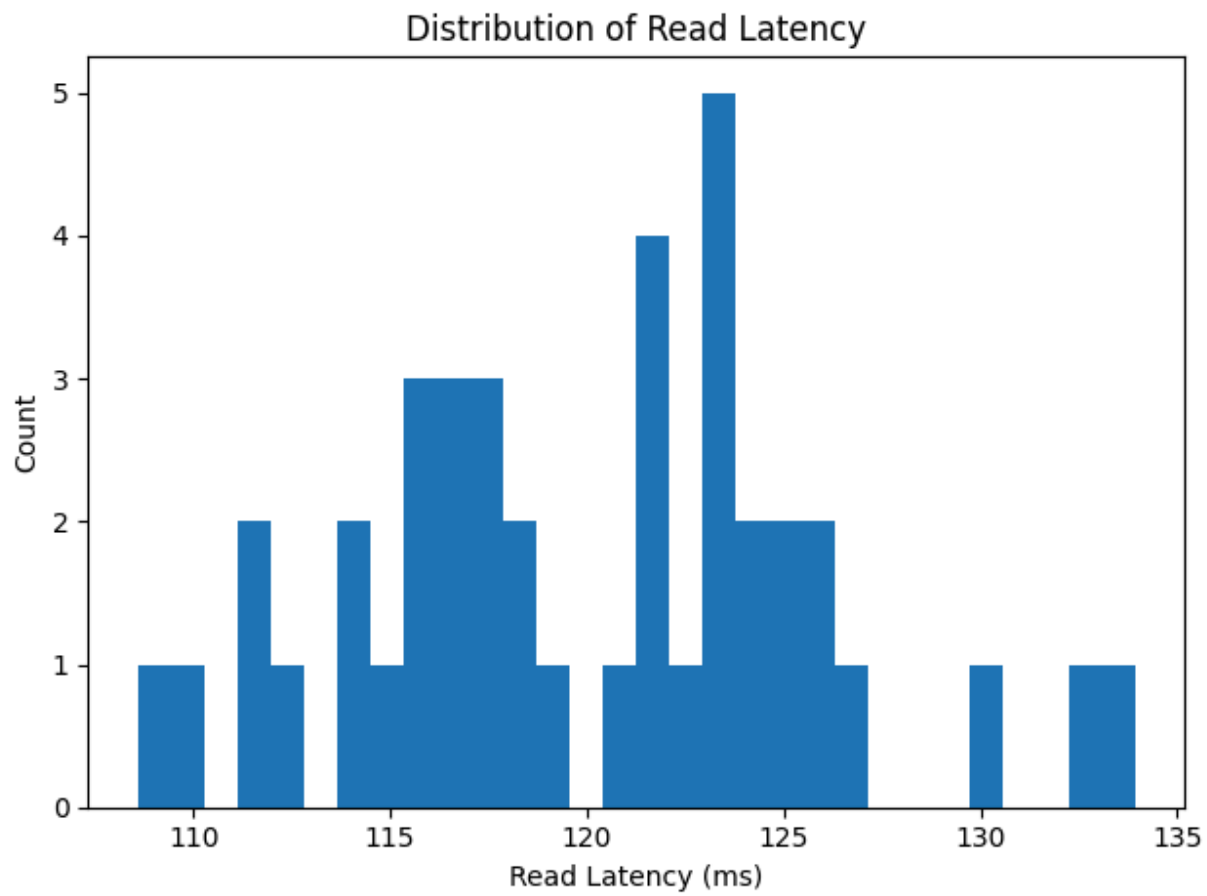


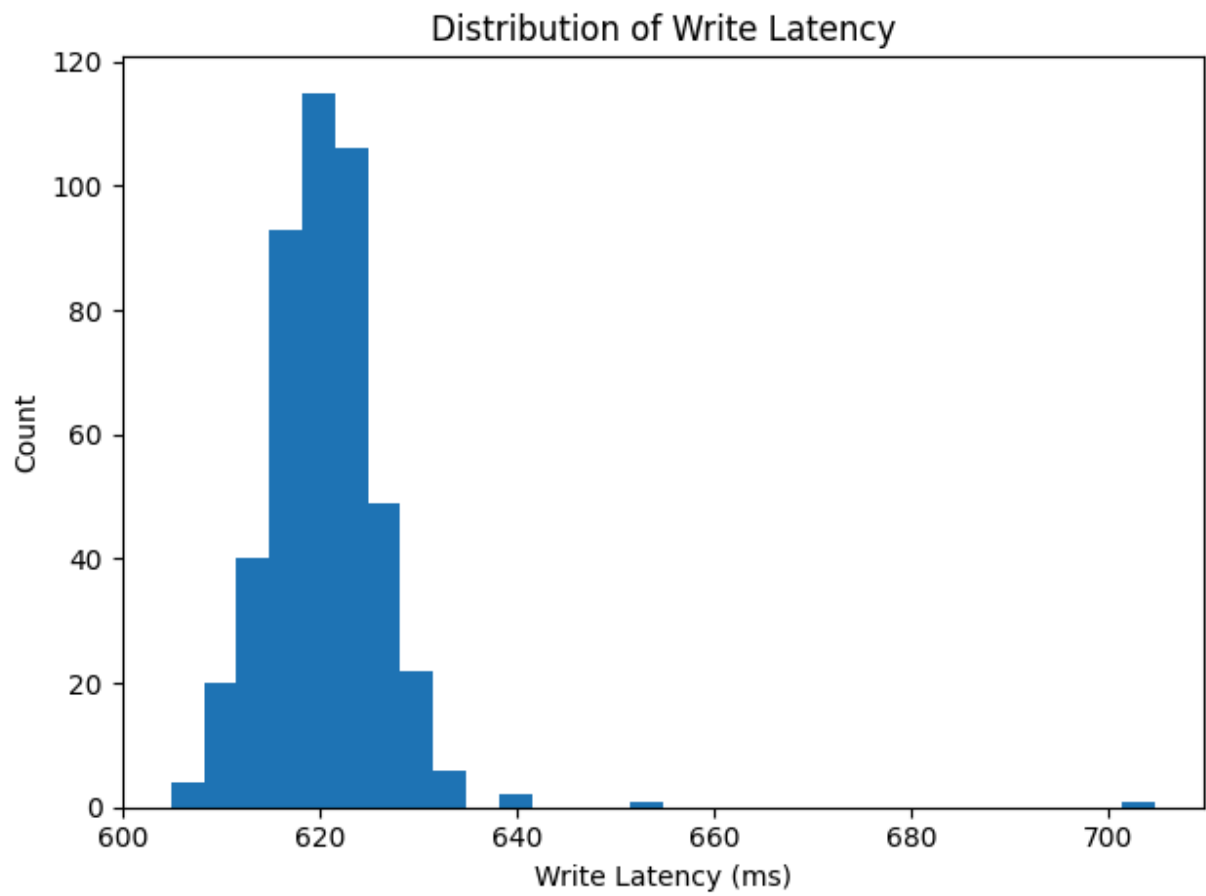
```
python load_tester/load_test.py --mode lf --strategy w3r3 --ops 500 --read_pct 10 --write_pct 90  
--outdir results/lf_w3r3_10r_90w
```

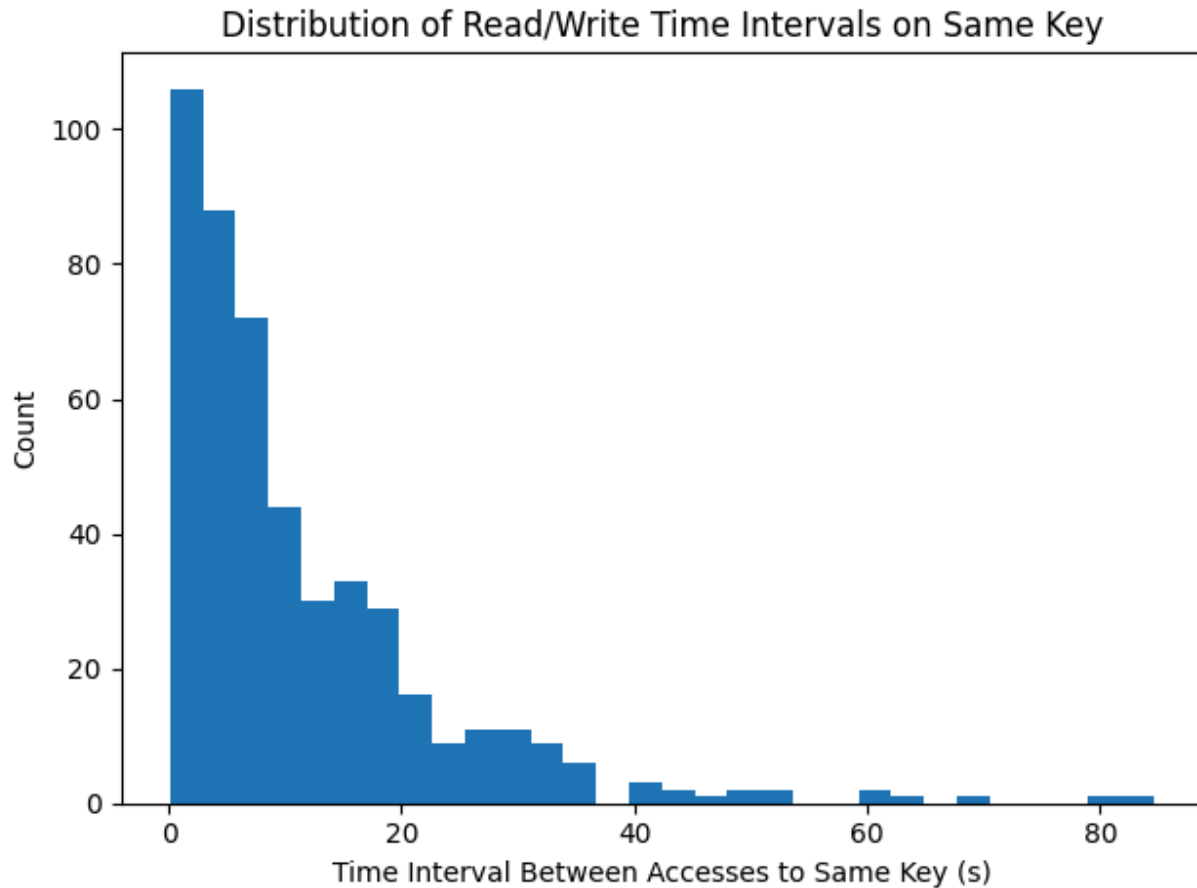
```
python load_tester/plot_results.py --indir results/lf_w3r3_10r_90w
```

HW10 > distributed-kv > results > lf_w3r3_10r_90w > [summary.json](#) > ...

```
1 {  
2   "mode": "lf",  
3   "strategy": "w3r3",  
4   "ops": 500,  
5   "read_pct": 10,  
6   "write_pct": 90,  
7   "stale_reads": 0,  
8   "total_reads": 41,  
9   "total_writes": 459,  
10  "avg_read_latency_ms": 120.10961463788479,  
11  "avg_write_latency_ms": 620.5545653597186  
12 }
```







Stop LF cluster

```
docker compose -f docker-compose-lf.yml down
```

Run Leaderless config

Start leaderless:

```
docker compose -f docker-compose-leaderless.yml up --build -d
```

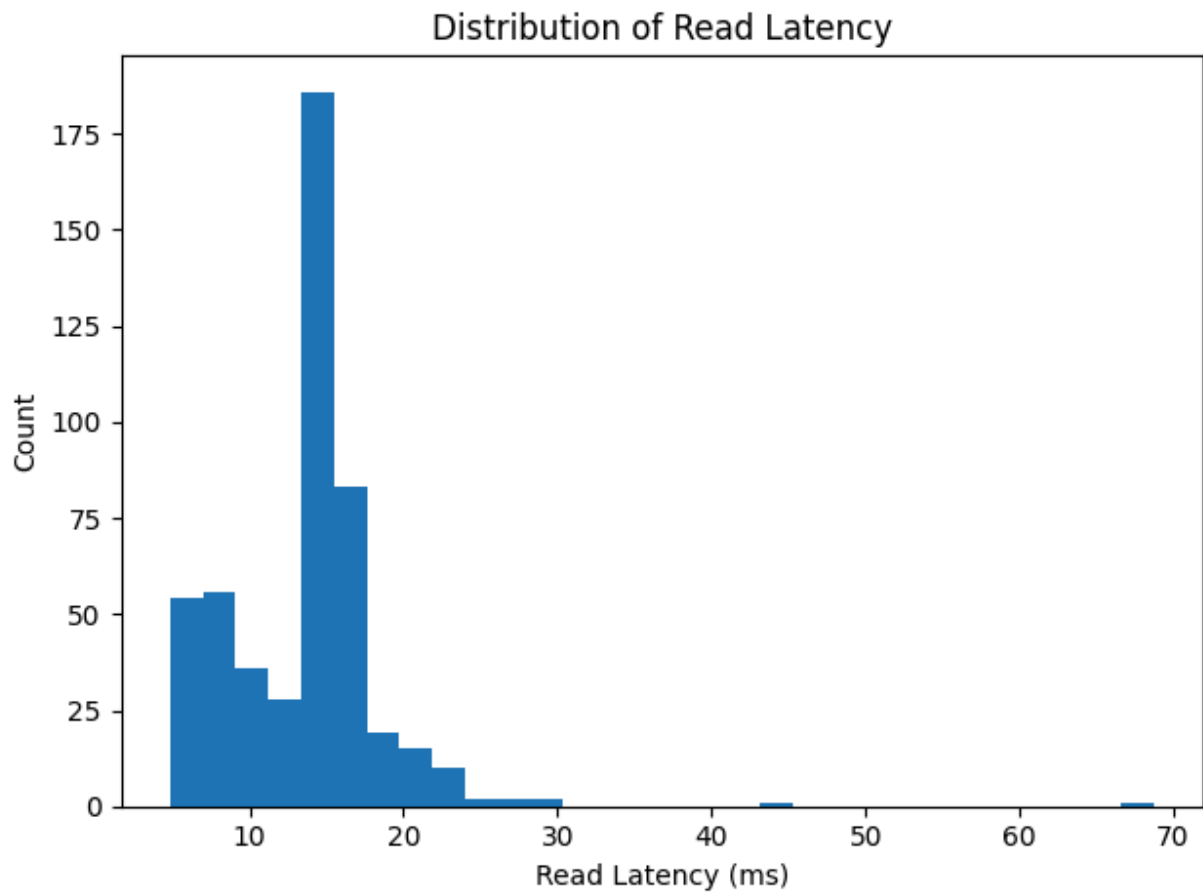
Run all 4 ratios:

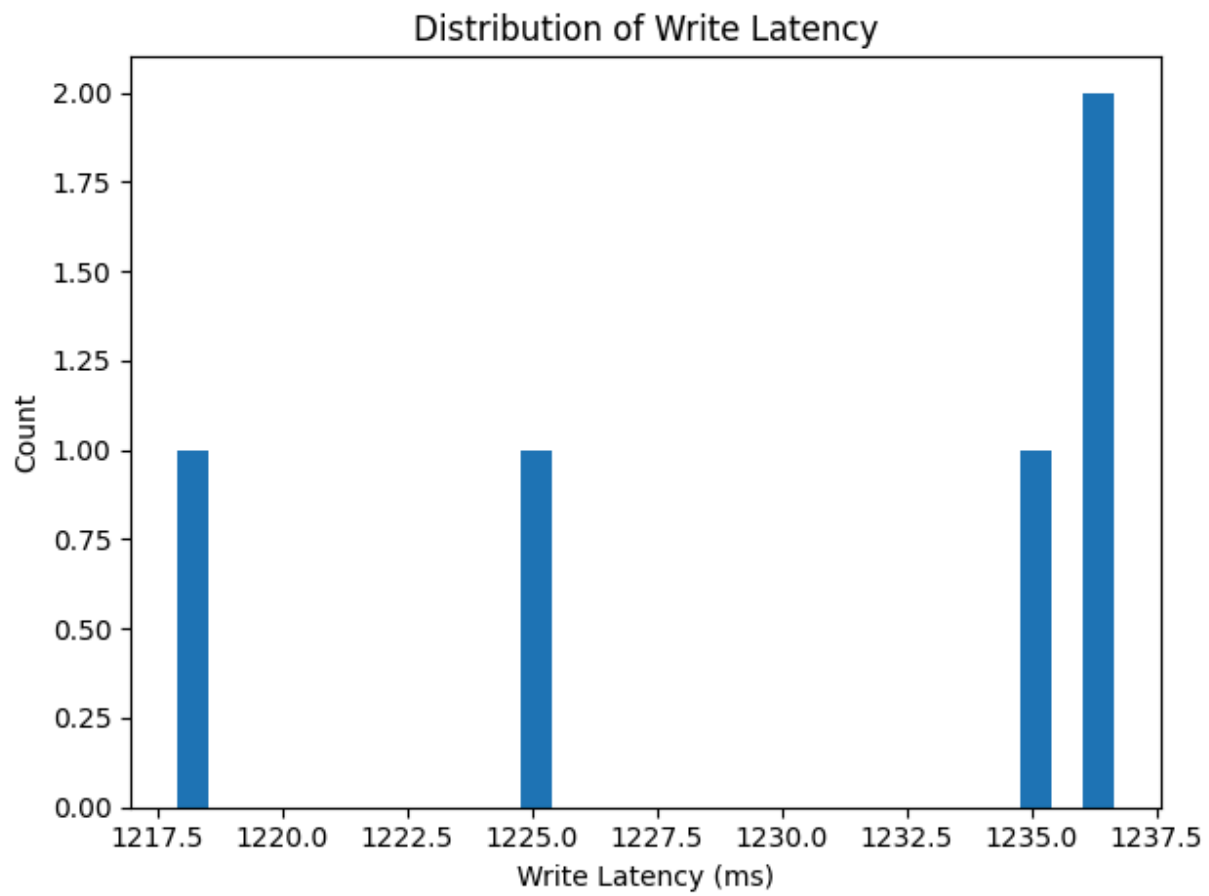
```
python load_tester/load_test.py --mode leaderless --strategy leaderless --ops 500 --read_pct 99
```

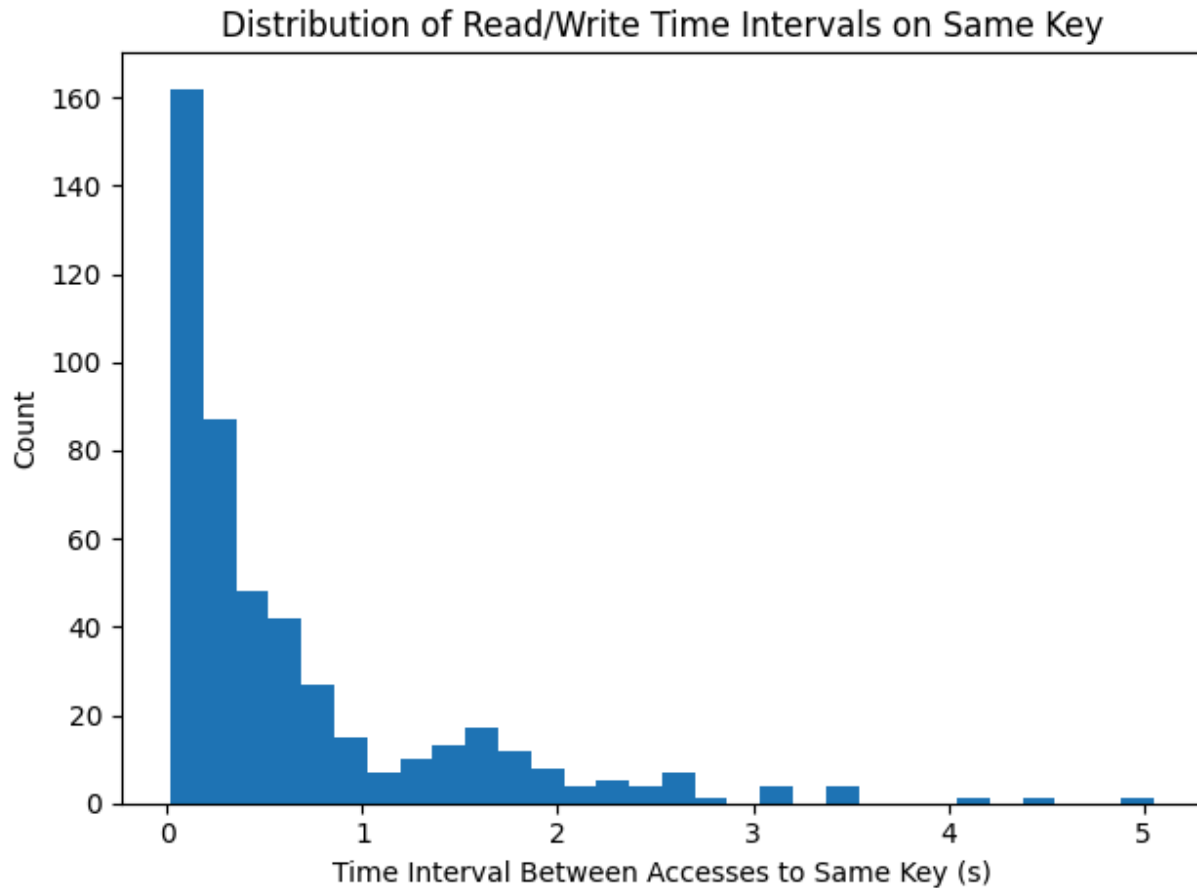
```
--write_pct 1 --outdir results/leaderless_99r_1w
```

```
python load_tester/plot_results.py --indir results/leaderless_99r_1w
```

```
1 {  
2   "mode": "leaderless",  
3   "strategy": "leaderless",  
4   "ops": 500,  
5   "read_pct": 99,  
6   "write_pct": 1,  
7   "stale_reads": 0,  
8   "total_reads": 495,  
9   "total_writes": 5,  
10  "avg_read_latency_ms": 13.476020806572503,  
11  "avg_write_latency_ms": 1230.1762800081633  
12 }
```







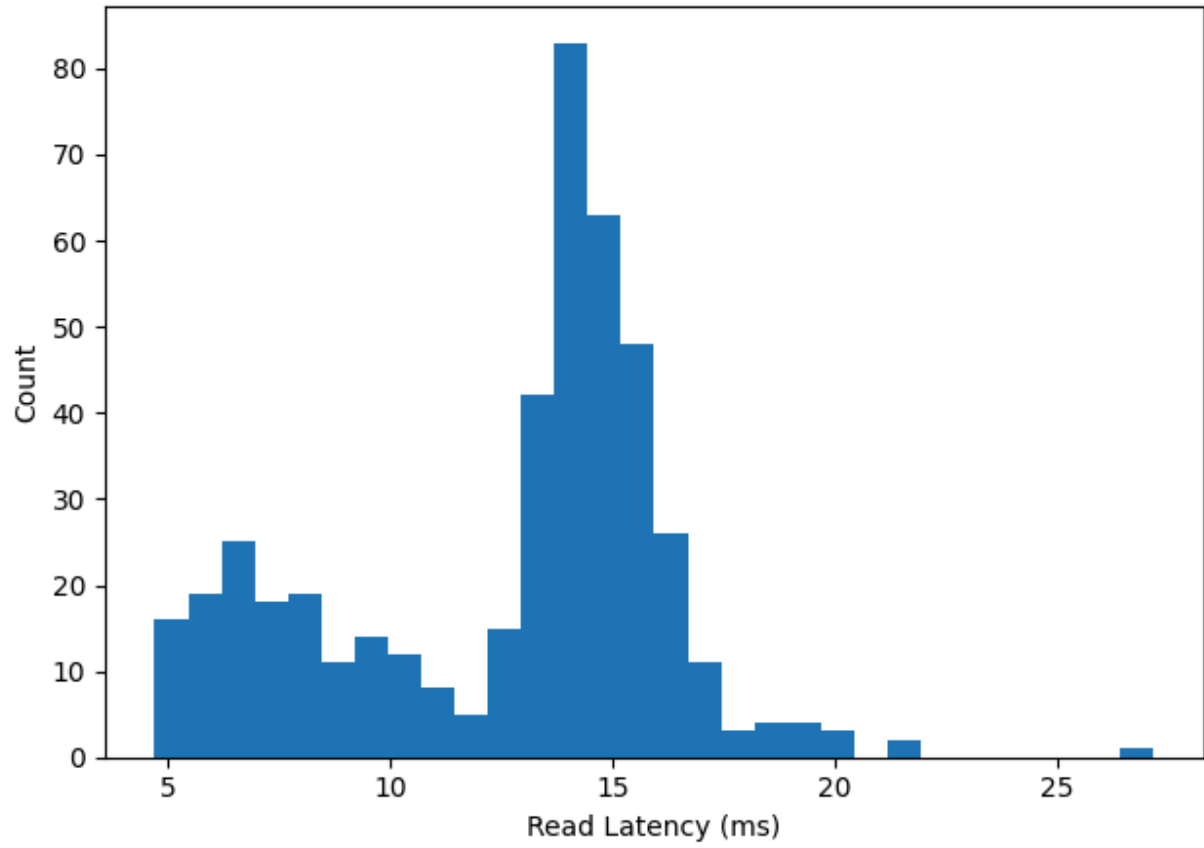
```
python load_tester/load_test.py --mode leaderless --strategy leaderless --ops 500 --read_pct 90  
--write_pct 10 --outdir results/leaderless_90r_10w
```

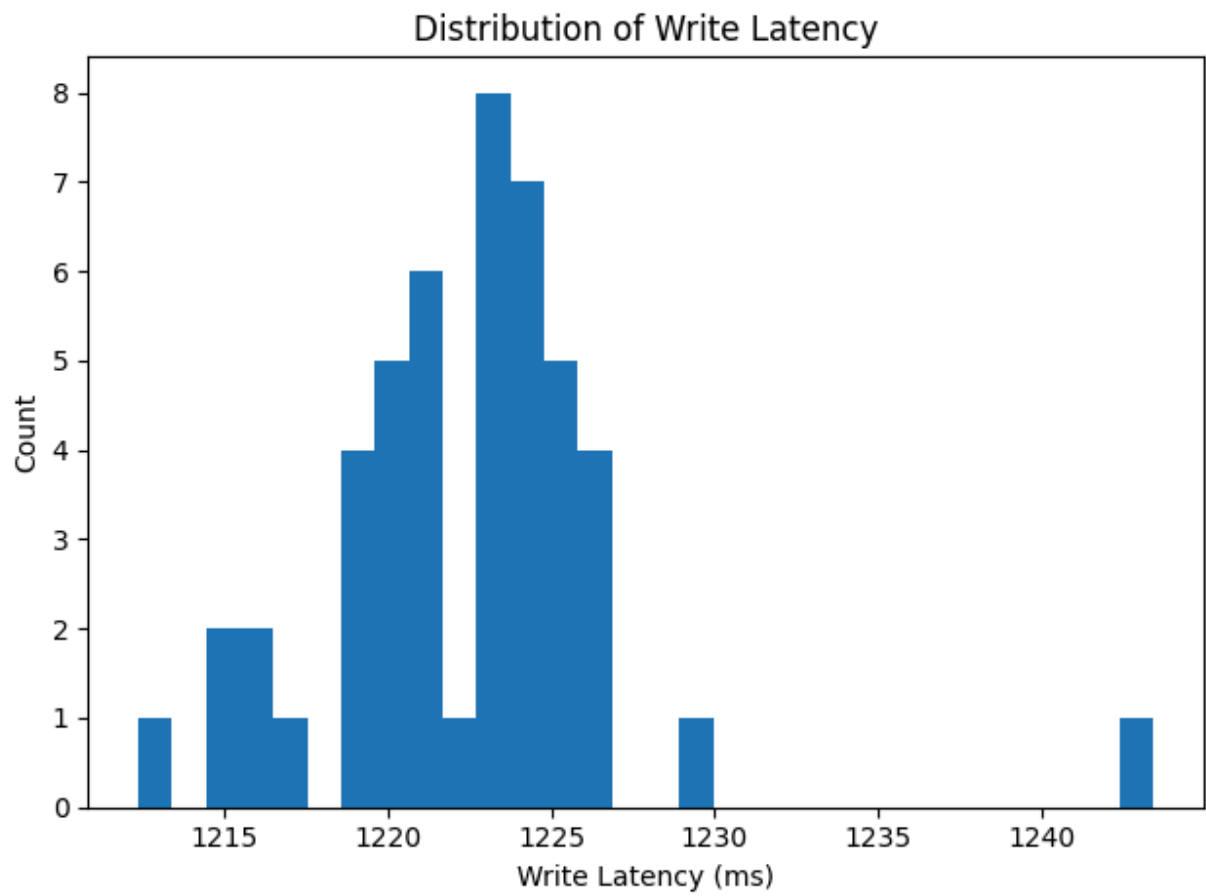
```
python load_tester/plot_results.py --indir results/leaderless_90r_10w
```

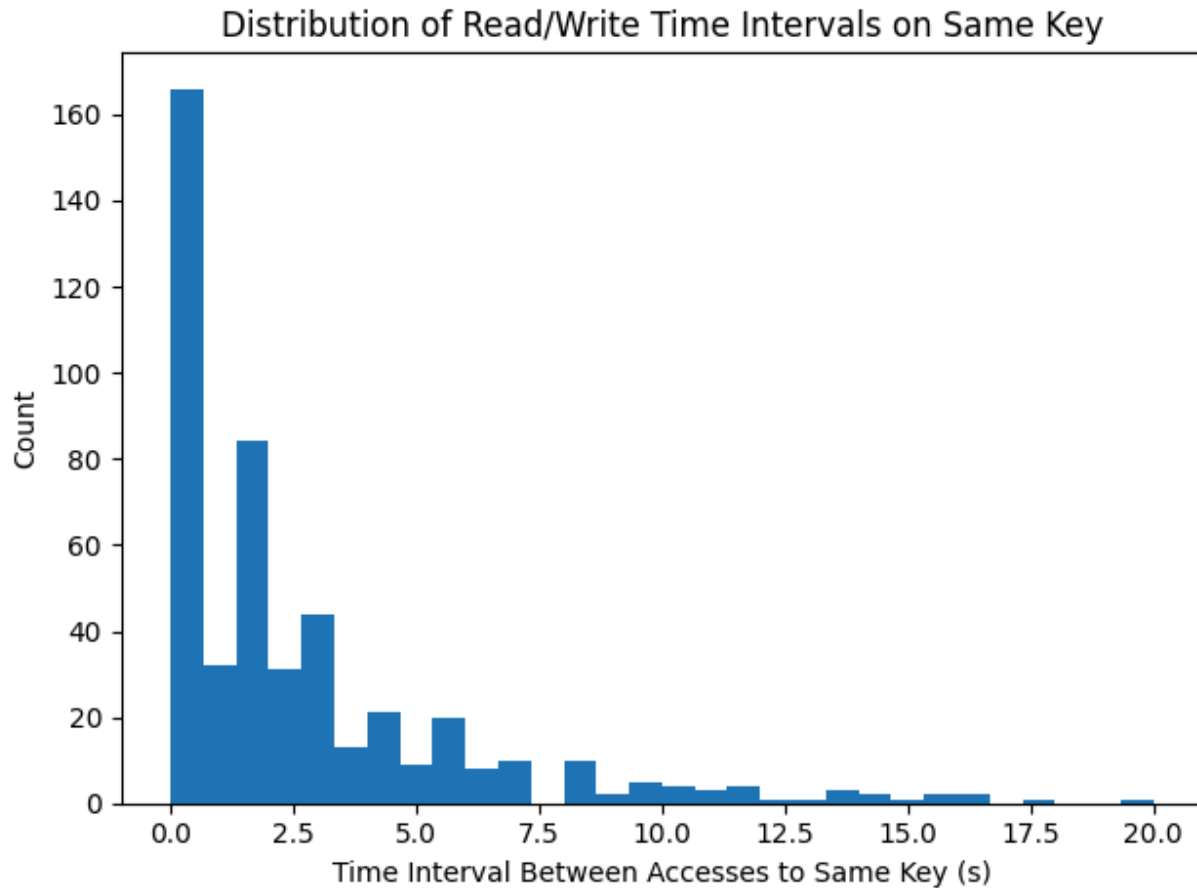
HW10 > distributed-kv > results > leaderless_90r_10w > summary.json > ...

```
1 {  
2   "mode": "leaderless",  
3   "strategy": "leaderless",  
4   "ops": 500,  
5   "read_pct": 90,  
6   "write_pct": 10,  
7   "stale_reads": 0,  
8   "total_reads": 452,  
9   "total_writes": 48,  
10  "avg_read_latency_ms": 12.569484292540857,  
11  "avg_write_latency_ms": 1222.460435417209  
12 }
```


Distribution of Read Latency





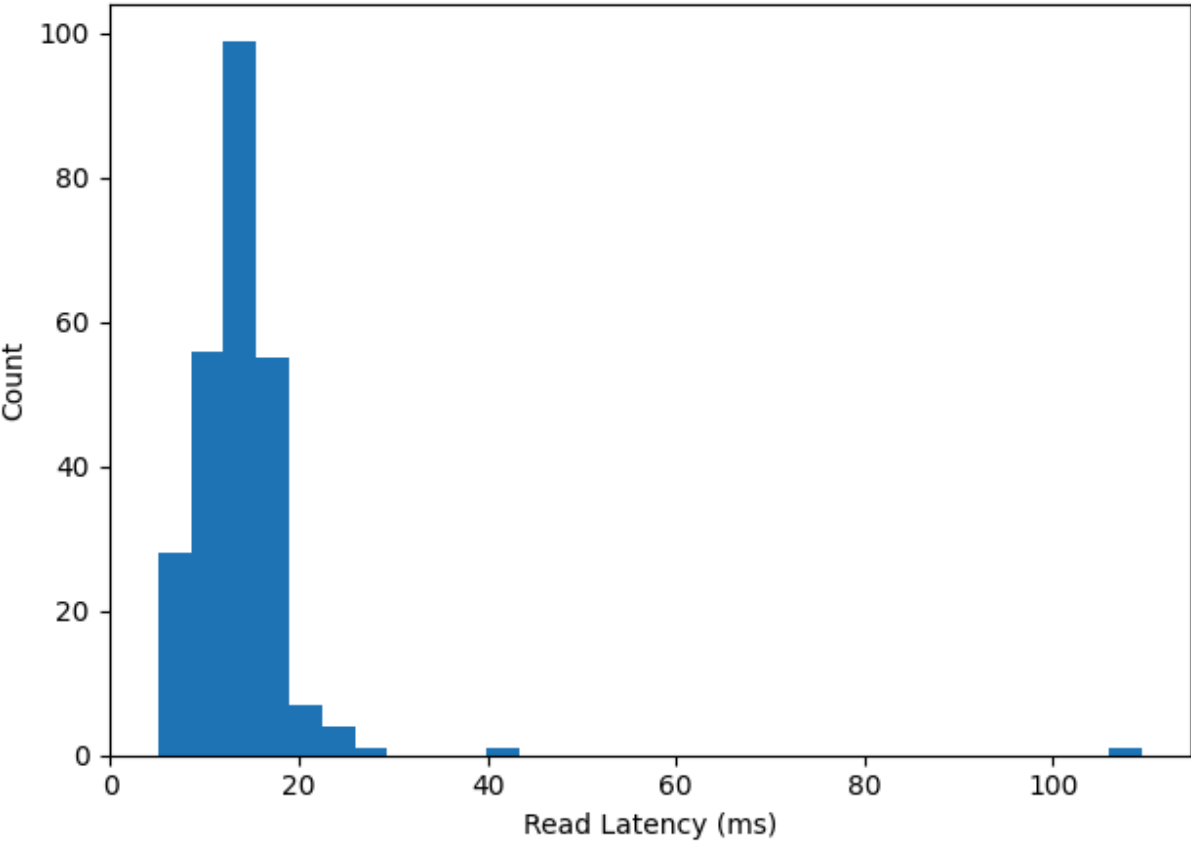


```
python load_tester/load_test.py --mode leaderless --strategy leaderless --ops 500 --read_pct 50  
--write_pct 50 --outdir results/leaderless_50r_50w  
python load_tester/plot_results.py --indir results/leaderless_50r_50w
```

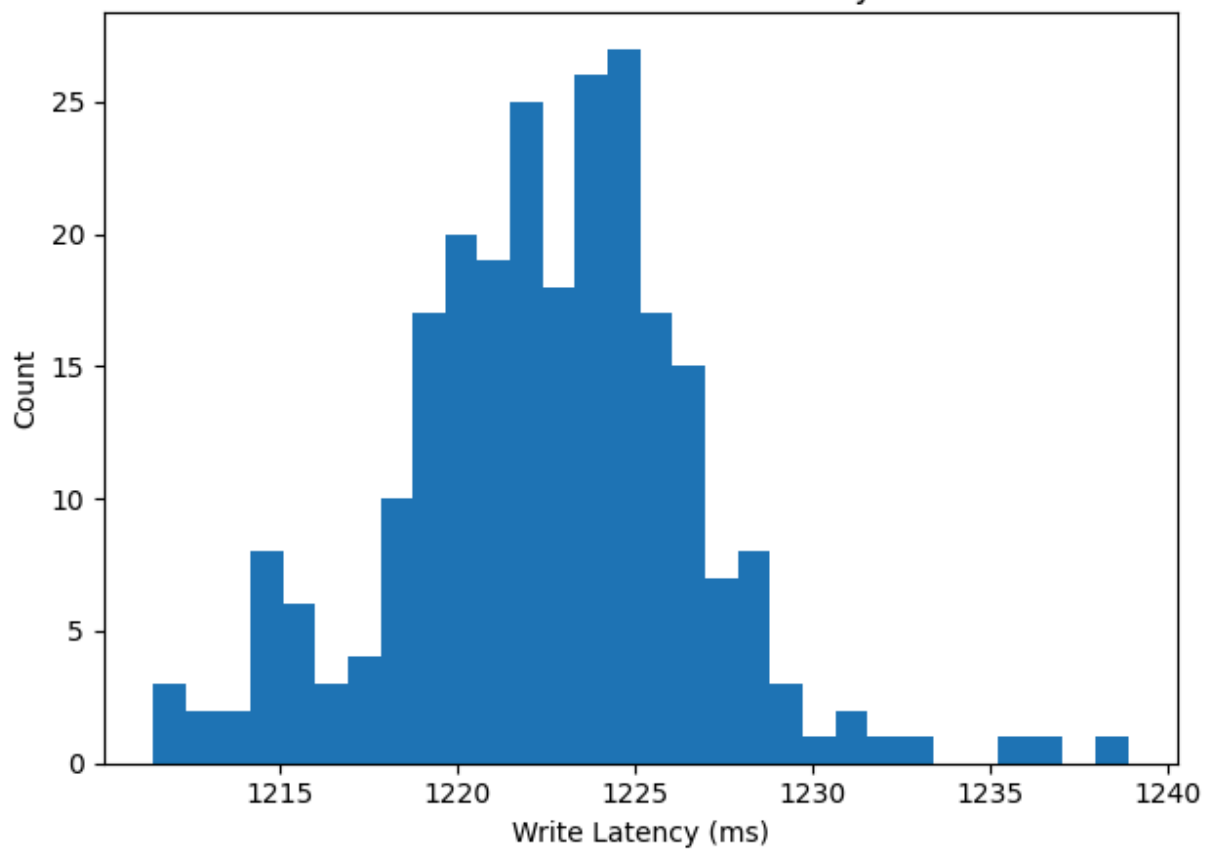
HW10 > distributed-kv > results > leaderless_50r_50w > {} summary.json > ...

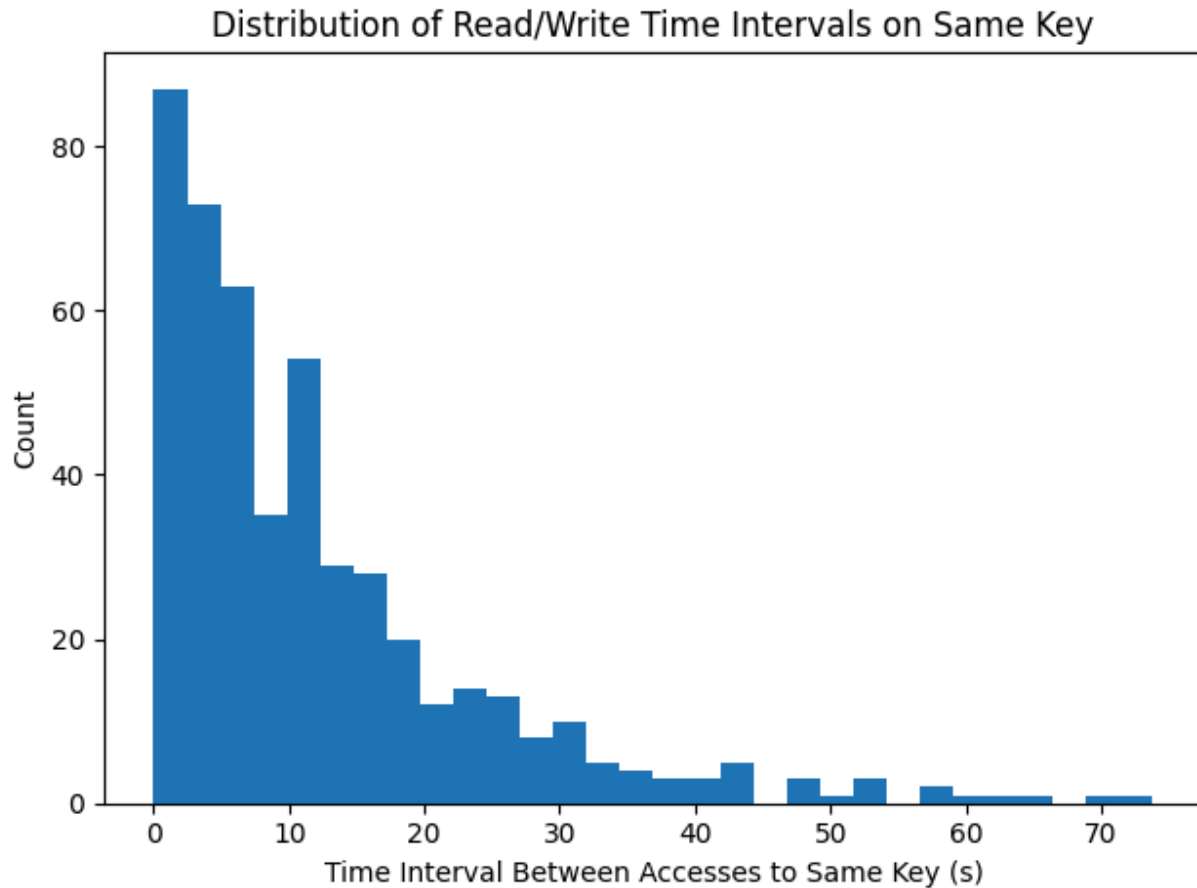
```
1  {  
2    "mode": "leaderless",  
3    "strategy": "leaderless",  
4    "ops": 500,  
5    "read_pct": 50,  
6    "write_pct": 50,  
7    "stale_reads": 0,  
8    "total_reads": 252,  
9    "total_writes": 248,  
10   "avg_read_latency_ms": 13.957766668191033,  
11   "avg_write_latency_ms": 1222.4900633053978  
12 }
```

Distribution of Read Latency



Distribution of Write Latency





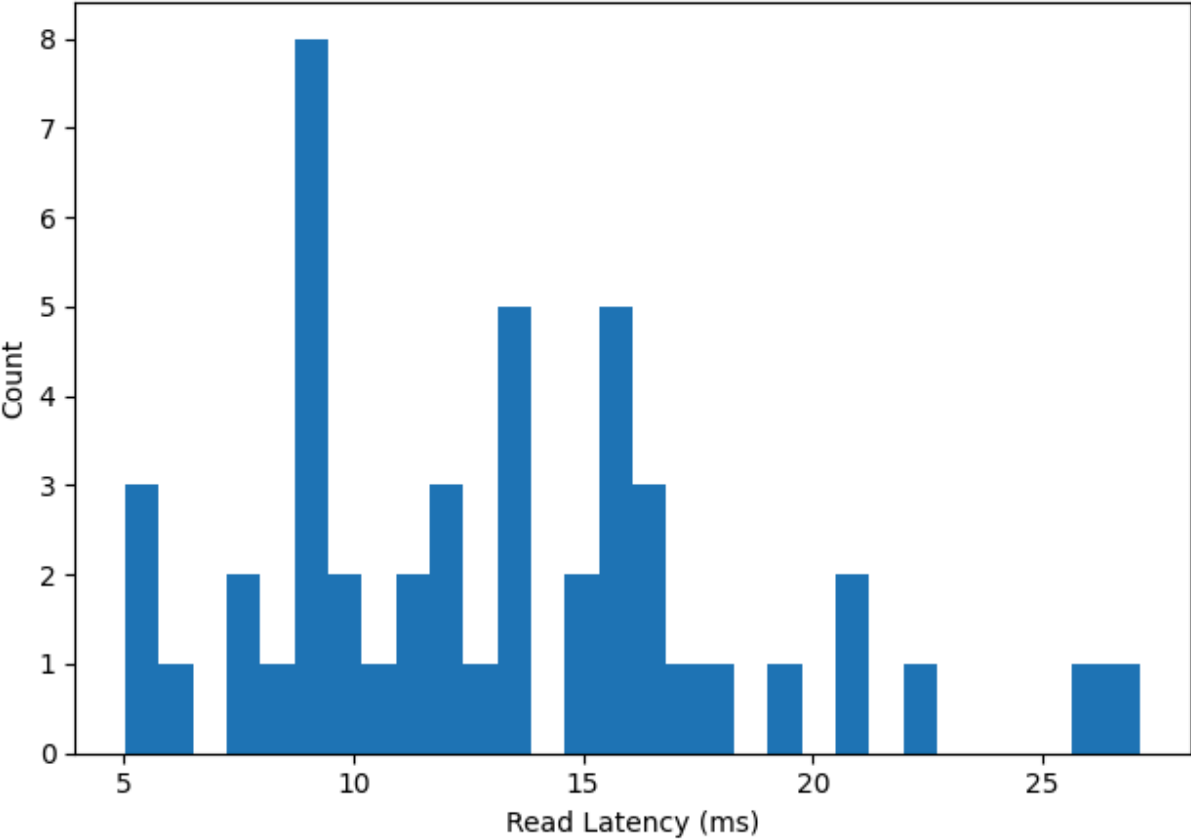
```
python load_tester/load_test.py --mode leaderless --strategy leaderless --ops 500 --read_pct 10  
--write_pct 90 --outdir results/leaderless_10r_90w
```

```
python load_tester/plot_results.py --indir results/leaderless_10r_90w
```

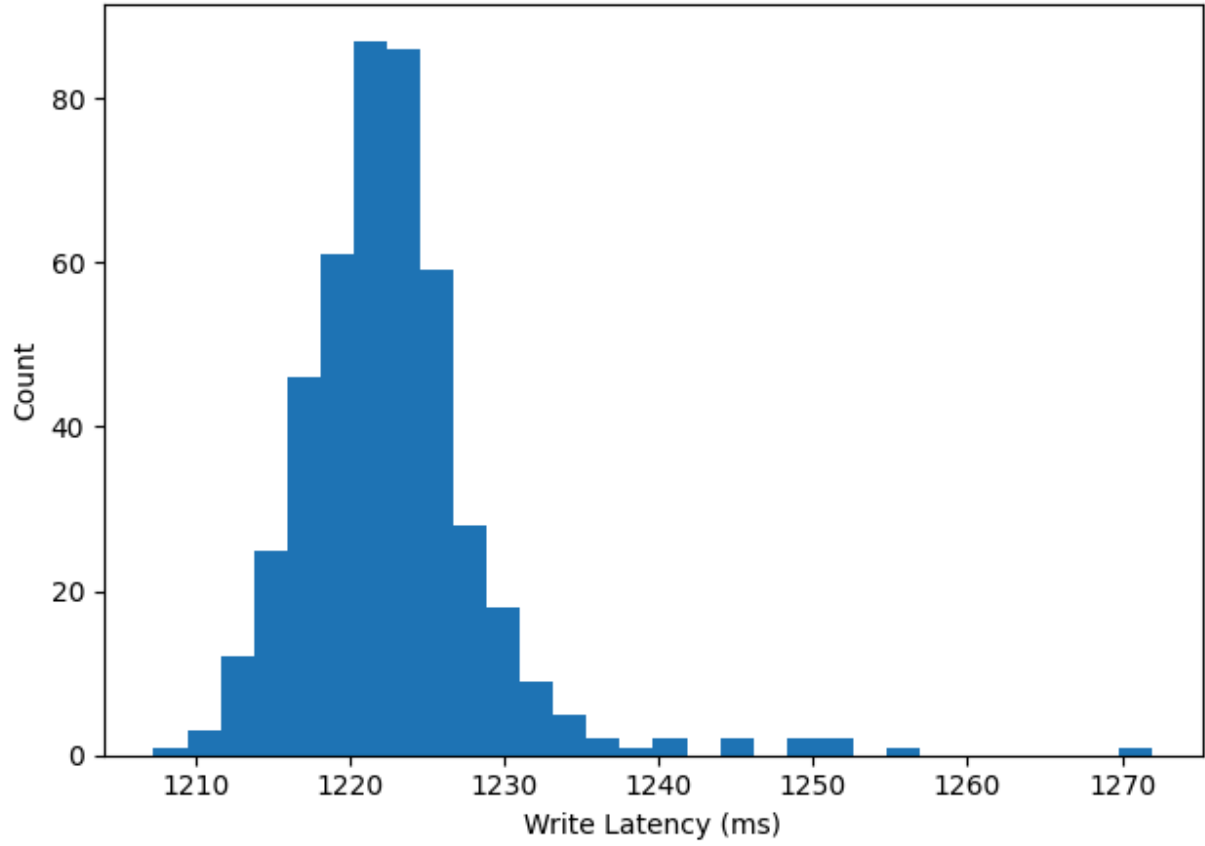
HW10 > distributed-kv > results > leaderless_10r_90w > {} summary.json > ...

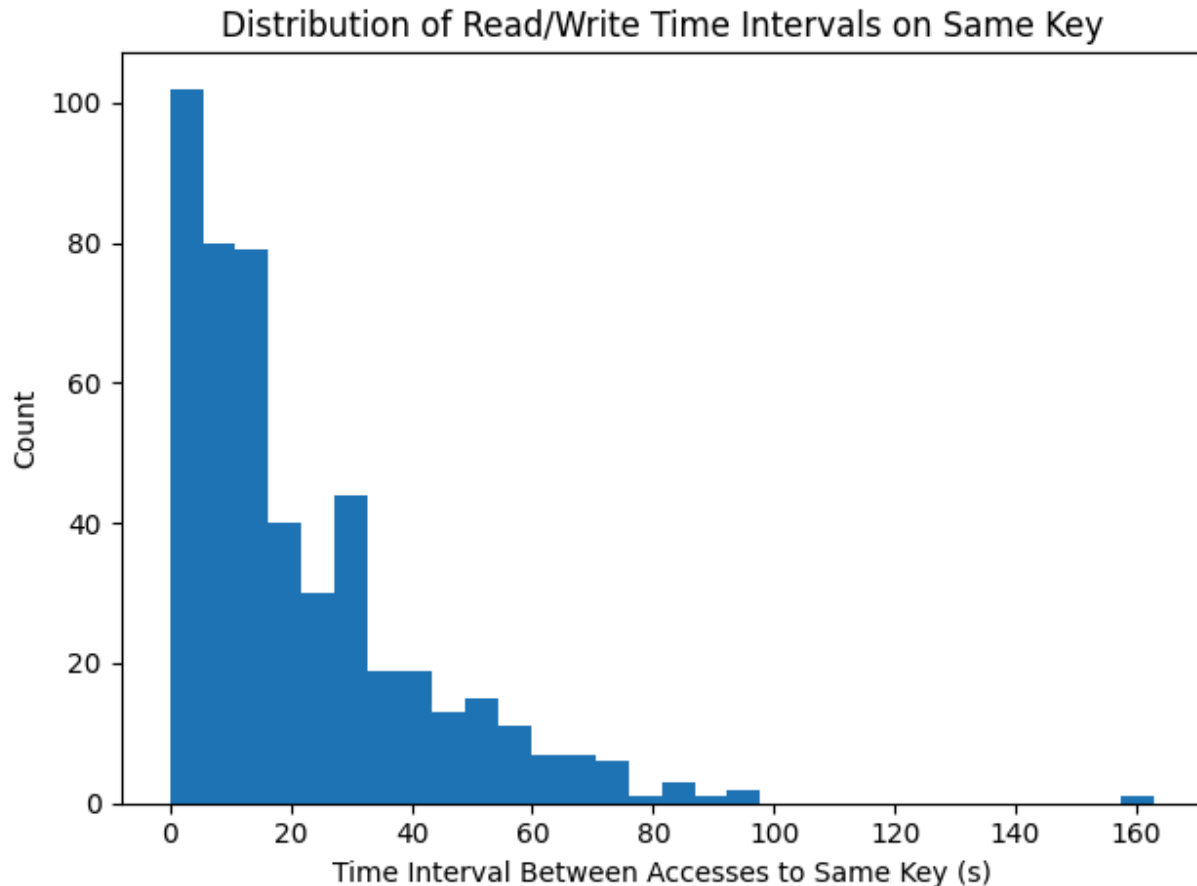
```
1 {  
2   "mode": "leaderless",  
3   "strategy": "leaderless",  
4   "ops": 500,  
5   "read_pct": 10,  
6   "write_pct": 90,  
7   "stale_reads": 0,  
8   "total_reads": 47,  
9   "total_writes": 453,  
10  "avg_read_latency_ms": 13.116757446117303,  
11  "avg_write_latency_ms": 1222.7124905060293  
12 }
```

Distribution of Read Latency



Distribution of Write Latency





Stop leaderless

`docker compose -f docker-compose-leaderless.yml down`

Discussion of Results

Looking across the results, the main pattern is that the tradeoff between read latency and write latency changes depending on how much write coordination is required before a response is returned.

For the Leader-Follower system with $W=5$, $R=1$, writes are expensive because the leader has to wait for all followers before acknowledging the write. That is why the write latency is much higher in those runs, especially compared with the strategies that allow earlier write acknowledgement. At the same time, reads stay fast because the read path is cheap when only one copy is needed.

For $W=1$, $R=5$, the opposite pattern appears. Writes are much faster because the leader can respond after only updating itself, but reads are much slower because each read has to gather values from every node and return the newest version. This is why the read latency becomes much larger in those runs even though write latency improves a lot.

For $W=3$, $R=3$, the system sits more in the middle. It does not push all the cost onto writes like $W=5$, $R=1$, and it does not push all the cost onto reads like $W=1$, $R=5$. Because of that, it behaves more like a compromise strategy.

Another important result is that the visible stale read counts in my shown runs were 0. I think this happened because the read logic in the tested paths often used either quorum logic or a leader-based path that returned a current version after acknowledgement, while the leaderless setup also waited for all replicas before acknowledging writes. That means the public API was more consistent after acknowledgement than the internal propagation window might suggest. So even though the system design allows inconsistency windows, my test runs did not expose many stale reads in the final recorded counts.

Overall, the results make sense with the system design: stronger write guarantees increase write latency, stronger read guarantees increase read latency, and balanced quorum settings spread the cost more evenly.

Which Leader-Follower Strategy Does Best for Each Read/Write Ratio?

99% reads / 1% writes

The best Leader-Follower strategy here is $W=5$, $R=1$. This is the most read-heavy workload, so it makes sense to optimize for cheaper reads even if writes are slower. Since writes are rare, paying a high write cost is less harmful than making every read expensive.

90% reads / 10% writes

I would still choose $W=5$, $R=1$ here, because the workload is still mostly dominated by reads. Read performance matters more than write speed in this case, and this strategy keeps the read side much cheaper.

50% reads / 50% writes

The most balanced choice is $W=3$, $R=3$. Since reads and writes matter about equally, it makes sense to use the strategy that splits the coordination cost across both sides instead of making one side extremely expensive.

90% writes / 10% reads

The best Leader-Follower strategy here is $W=1$, $R=5$. This workload is dominated by writes, so the system benefits from returning quickly on writes and accepting the higher cost on the less frequent reads.

In short, $W=5$, $R=1$ fits read-heavy workloads, $W=1$, $R=5$ fits write-heavy workloads, and $W=3$, $R=3$ is the balanced middle option.

Which Kind of Database Would Be Best for What Kind of Application?

The best database design depends on what the application cares about most.

A Leader-Follower database with $W=5$, $R=1$ is a good fit for applications where reads happen much more often than writes, and where it is important that once a write is acknowledged, the system has already pushed that value everywhere. This could fit systems where users mostly read data and only occasionally update it.

A Leader-Follower database with $W=1$, $R=5$ is a better fit for applications where fast write acknowledgement matters more than cheap reads. This could fit systems where many updates are happening constantly, but it is acceptable for reads to pay more coordination cost in order to get the newest value.

A Leader-Follower database with $W=3$, $R=3$ is a better fit for balanced workloads where both reads and writes matter and neither one should become extremely expensive. This is the most reasonable middle-ground design when traffic patterns are mixed.

A Leaderless database with $W=N$ and $R=1$ is useful when there is no single distinguished leader and the system wants decentralized write coordination. That can be attractive in systems that want to avoid depending on one central writer, but the tradeoff is that stale reads can happen more easily during propagation windows if clients read from a node that has not been updated yet.

So there is no single best design for everything. The right choice depends on whether the application is mostly read-heavy, write-heavy, balanced, or more decentralized.